

1 A Methods Atlas on the Ames Housing Dataset

1.0.1 MATH 4230 — Applied Statistical Methods for Data Science

1.0.2 Final Capstone Project

Student: Krrithik Ezhilarasan

Course: MATH 4230 — Applied Statistical Methods for Data Science, Section 01

Instructor: Dr. Anjana Yatawara

Institution: California State University, Bakersfield

Semester: Spring 2026

Submission Date: May 18, 2026

1.1 Abstract

This report applies thirteen statistical learning methods from the syllabus to a single dataset — the Ames Housing dataset (2,925 observations on 80 property features) — and synthesizes the results into a cross-method comparison. The methods include simple and multiple linear regression, logistic regression, cross-validation and the bootstrap, ridge and lasso regularization, decision trees, bagging, random forest, and gradient boosting ensembles, support vector machines, K-nearest neighbors, principal component analysis with principal components regression, and a feedforward neural network.

Five prediction questions were posed: a primary regression task on log sale price (answered by eight methods), a binary classification task on above-median pricing (answered by two methods), and three standalone questions on finished living space, three-class price tier classification, and the presence of a fireplace. Principal component analysis was applied as an unsupervised exploration of the predictor structure.

Three findings recur across the methods. First, the best test RMSE on log sale price converges to a narrow \$19,900–\$21,400 band across seven independent model families, suggesting a structural ceiling imposed by the data rather than by any single method’s capability. Second, four independent feature-selection methods — BIC stepwise selection, lasso L1 shrinkage, random forest importance, and gradient boosting importance — converge on the same six predictors as the genuine drivers of price: Overall Quality, Above-Grade Living Area, Year Built, Total Basement Square Footage, Garage Capacity, and Garage Area. Third, on tabular data at this sample size, well-regularized linear methods (ridge, lasso) are competitive with both gradient boosting and a tuned neural network. Ridge regression is recommended as the deployable model: test RMSE of \$19,900, training time under three seconds, fully interpretable coefficients.

1.2 Table of Contents

Chapter 1 — Project Framing and Dataset Selection

Chapter 2 — Dataset Profile & Exploratory Data Analysis

Chapter 3 — Simple Linear Regression

How accurately can sale price be predicted from above-grade living area alone?

Chapter 4 — Multiple Linear Regression

Does adding all 207 features substantially improve prediction over a single predictor — and how should the resulting multicollinearity be handled?

Chapter 5 — Logistic Regression

Can a regularized logistic regression accurately classify homes as above or below median sale price?

Chapter 6 — Cross-Validation and the Bootstrap

Which resampling methods give the most stable estimate of test error, and how does the bootstrap compare to analytic standard errors?

Chapter 7 — Ridge and Lasso Regression

When predictors are highly correlated, how do Ridge and Lasso compare to OLS — and to each other — on accuracy and parsimony?

Chapter 8 — Decision Trees

Two parallel analyses: predicting total finished living space from lot/site features (regression tree), and classifying homes into Budget, Mid-tier, or Premium price segments (classification tree).

Chapter 9 — Tree Ensembles: Bagging, Random Forest, and Boosting

Can tree-based ensembles overcome the high-variance limitation of a single tree and capture non-linearities the linear models cannot?

Chapter 10 — Support Vector Machines

Can a kernel-based SVM accurately classify whether a home has a fireplace — and does the RBF kernel justify its added complexity over a linear kernel?

Chapter 11 — K-Nearest Neighbors

Can a non-parametric KNN approach match the parametric logistic regression of Chapter 5, and what does the optimal K reveal about the locality of price structure?

Chapter 12 — Principal Component Analysis

How many underlying dimensions of variation are present in the Ames numeric predictors, and does PCR preserve enough information for accurate prediction?

Chapter 13 — Neural Networks

Does a feedforward neural network meaningfully improve on the linear and tree-based methods of earlier chapters, or does the limited sample size cap its performance?

Chapter 14 — Synthesis and Cross-Method Comparison

Chapter 15 — Reflection

References

Appendices

1.3 How to Read This Document

This report is organized as fifteen chapters plus front and back matter, designed to be read either sequentially or by jumping to specific chapters of interest. Each method chapter (Chapters 3–13) follows an identical nine-section template, so a reader looking for a specific kind of information —

the research question, the methodological justification, the numerical findings, or the real-world decision — can navigate directly to that section in any chapter:

1. **The Question** — the specific real-world prediction problem the chapter answers.
 2. **Method in Brief** — a plain-language explanation of how the method works.
 3. **Why This Method?** — methodological justification for applying this method to this question.
 4. **Data Setup** — the response variable, predictors, splits, scaling, and tuning protocol.
 5. **Analysis** — the actual model fit and the relevant diagnostic plots.
 6. **Diagnostics & Tuning** — hyperparameter selection and assumption checks.
 7. **Findings** — the numerical results.
 8. **Real-World Decision** — what a stakeholder should do with these results.
 9. **Caveats & Limitations** — what the method cannot tell us, and why.
-

2 Chapter 1 — Introduction and Statistical Thinking

2.1 §1.1 Project Motivation

Residential real-estate pricing depends on many features at once. A home’s value reflects its size, quality, age, location, condition, and amenities, and these features interact in ways that are not obvious from looking at any single variable. A real assessment combines all of this information using experience and judgment.

This project uses the Ames Housing dataset to compare every statistical learning method learnt in my class MATH 4230 on the problem of explaining and predicting home sale prices. The goal here is to use the available property-level features to compare the main methods covered in the course and explain what each method adds.

2.2 §1.2 Main Question and Responses

The main project question is:

Can property-level features explain home sale prices in Ames, Iowa — and which methods produce the best balance of accuracy and interpretability?

This gives the project both a regression problem and a classification problem. The regression response is $\log(\text{SalePrice})$, the natural logarithm of the sale price in dollars. The classification response is **AboveMedian**, which is 1 if the home sold for more than the training-set median (\$160,000) and 0 otherwise.

Two additional targets are derived from the data for specific chapters:

- `PriceTier` (3-class: Budget, Mid, Premium) for the classification tree in Chapter 8
- `HasFireplace` (binary) for the SVM analysis in Chapter 10
- `TotalFinishedSpace` (continuous, in square feet) for the regression tree in Chapter 8

2.3 §1.3 Research Questions

The chapter questions are:

1. How strongly does above-grade living area alone explain sale price?
2. How well do multiple predictors explain sale price together?
3. Can a logistic regression classify above-median priced homes?
4. How reliable are models under cross-validation?
5. How uncertain are key estimated effects under the bootstrap?
6. Does ridge regression improve over ordinary least squares?
7. Which variables does lasso keep?
8. What rules do regression and classification trees find?
9. How much living space does a buyer expect from a given lot?
10. Do ensemble methods (bagging, random forest, boosting) improve prediction?
11. Can an SVM classify whether a home has a fireplace?
12. Does the RBF kernel improve over a linear kernel?
13. Can KNN match logistic regression on the same binary target?
14. How many underlying dimensions explain the variation in numeric predictors?
15. Does a neural network outperform the linear and tree-based methods?
16. Which model should a real-estate analytics team actually deploy?"

2.4 §1.4 Train-Test Split

A single 80/20 train/test split is created in Chapter 2 and reused unchanged across every modeling chapter. The split is stratified on the binary `AboveMedian` indicator to preserve class balance in both partitions, with `random_state = 42` for reproducibility. This produces 2,340 training observations and 585 test observations.

A stratified random split is appropriate here because Ames home sales are independent observations — unlike repeated measurements on the same property over time, each row records a separate transaction. The test set is held out from the moment the split is created and is not used for imputation, scaling, hyperparameter tuning, or any other data-dependent decision in any chapter until final model evaluation.

2.5 §1.5 Statistical Thinking Statement

Home sale prices have a lot of natural variation. They depend on features the dataset captures — size, quality, year built, lot size, neighborhood, garage capacity, basement finish — but also on features the dataset does not capture, like interior condition at the time of sale, the negotiating skill of the buyer and seller, motivated-seller circumstances, and market conditions at the moment of the transaction. Since the dataset does not include every relevant factor, I do not expect any single model to explain everything.

The goal is to compare methods honestly, identify which features multiple methods independently agree are important, and decide which methods are most useful for interpretation, prediction, or both.

3 Chapter 2 — Dataset Profile & Exploratory Data Analysis

Before modeling, the dataset was checked for structure, missingness, class balance, and obvious patterns. This chapter documents the data preparation steps that every subsequent chapter relies on.

3.1 §2.1 Dataset Source and Provenance

The dataset is the **Ames Housing Dataset**, compiled by Dean De Cock of Truman State University in 2011 as a teaching alternative to the older Boston Housing dataset. The original data comes from the Ames, Iowa Assessor’s Office, recording tax assessment records for residential property sales between 2006 and 2010 [1]. The dataset was accessed via Kaggle (user **shashanknecrothapa**) on May 16, 2026.

The data was originally collected for property-tax assessment, which makes it well-suited for predicting market sale prices — the variables recorded are exactly the kind a buyer would evaluate before purchase. The study is observational and cross-sectional: each row is one house sale, with no repeated measurements over time. The full dataset contains **2,930 observations across 82 variables**.

The dataset supports both response types required by the project. The continuous regression target is **SalePrice** (in U.S. dollars), and a binary classification target — **AboveMedian** — is derived in §2.5.

3.2 §2.2 Main Variables

The 82 variables fall into four categories per De Cock’s documentation:

Type	Count	Examples
Nominal categorical	23	Neighborhood, MS Zoning, Bldg Type, Sale Condition
Ordinal categorical	23	Overall Qual, Exter Qual, Kitchen Qual, Bsmt Cond
Discrete numeric	14	Year Built, Full Bath, Garage Cars, Bedroom AbvGr
Continuous numeric	20	SalePrice, Gr Liv Area, Total Bsmt SF, Lot Area
Identifier (non-predictor)	2	Order, PID

Table 2.1 lists the variables most relevant to the modeling chapters. The complete codebook is available in De Cock (2011).

Table 2.1: Main variables used across the modeling chapters.

Variable	Type	Description
SalePrice	Continuous	Final sale price in USD (regression response)
Gr Liv Area	Continuous	Above-grade living area, sq ft

Variable	Type	Description
Total Bsmt SF	Continuous	Total basement area, sq ft
1st Flr SF, 2nd Flr SF	Continuous	First- and second-floor square footage
Lot Area	Continuous	Lot size, sq ft
Overall Qual	Ordinal (1–10)	Material and finish quality rating
Overall Cond	Ordinal (1–10)	Overall condition of the house
Year Built	Discrete	Original construction year
Year Remod/Add	Discrete	Year of last remodel
Garage Cars / Garage Area	Discrete / Cont.	Garage capacity and size
Full Bath	Discrete	Full bathrooms above grade
Neighborhood	Nominal (28 levels)	Named neighborhood within Ames
Sale Condition	Nominal (6 levels)	Whether sale was normal, partial, abnormal, etc.

3.3 §2.3 Summary Statistics

Table 2.2 summarizes the distributions of the most important continuous and discrete predictors in the raw dataset. The full descriptive statistics for all 37 numeric variables are computed in the accompanying notebook and stored for reference; only the most relevant variables appear here.

Table 2.2: Summary statistics for key numeric variables, sorted alphabetically. Mean, median, and standard deviation are reported on the original (un-logged) scale.

Variable	n	Mean	Median	Std Dev	Min	Max
Fireplaces	2,930	0.60	1.00	0.65	0	4
Full Bath	2,930	1.57	2.00	0.55	0	4
Garage Cars	2,929	1.77	2.00	0.76	0	5
Gr Liv Area (sq ft)	2,930	1,499.69	1,442.00	505.51	334	5,642
Lot Area (sq ft)	2,930	10,148	9,437	7,880	1,300	215,245
Overall Qual (1–10)	2,930	6.09	6.00	1.41	1	10
SalePrice (\$)	2,930	180,796	160,000	79,887	12,789	755,000
Total Bsmt SF	2,929	1,051.61	990.00	440.62	0	6,110
TotRms AbvGrd	2,930	6.44	6.00	1.57	2	15
Year Built	2,930	1,971	1,973	30.25	1872	2010

Three observations from this summary influence later modeling choices. First, **SalePrice** ranges over nearly two orders of magnitude (\$12,789 to \$755,000), motivating the log transformation in

§2.5. Second, **Lot Area** has an extremely long right tail (maximum 215,245 against a median of 9,437), reflecting a small number of atypical agricultural-zoned parcels in the dataset. Third, the median home was built in 1973 and is roughly 1,442 sq ft above grade — useful anchor points for interpreting effect sizes in later chapters.

3.4 §2.4 Missing Values and Their Handling

The raw dataset contains missing values in 27 of its 82 columns. The key insight from De Cock’s documentation is that **most NaN values are not genuinely missing — they encode the absence of a feature on the property**. For example, `Pool QC = NaN` means the home has no pool, not that the pool’s quality is unknown.

Three patterns of missingness are present, summarized in Table 2.3.

Table 2.3: Missingness patterns and how each is handled.

Pattern	Affected columns	Example	Handling
Structural absence	22 columns	Pool QC (99.6% NaN — no pool), Garage * (5.4% NaN — no garage), Bsmt * (~3% NaN — no basement)	Recode as explicit labels (“No Pool”, “No Garage”, “No Basement”) or zeros
Genuine missingness	2 columns	Lot Frontage (16.7% NaN), Electrical (1 row)	Impute on training-set statistics only — neighborhood median for Lot Frontage, mode for Electrical
Boundary cases	A few cells	Bsmt square-footage cells where the row has no basement	Set to zero, consistent with structural handling

After this recoding, no missing values remain in any modeling variable. All imputation was performed using statistics computed on the training set only, after the train/test split, to prevent leakage of test-set information.

No rows were dropped due to missingness. Dropping any row with any NaN would have eliminated nearly the entire dataset, since `Pool QC` alone is 99.6% NaN.

3.5 §2.5 The Response Variable: SalePrice

Real-estate sale prices are typically right-skewed: most homes cluster around a moderate central price with a long tail of expensive properties. The raw **SalePrice** distribution shows this clearly with a skewness of 1.59. A natural-log transformation brings the skewness to -0.04 , producing a near-symmetric distribution suitable for the linear-regression assumptions used in Chapters 3, 4, and 7.

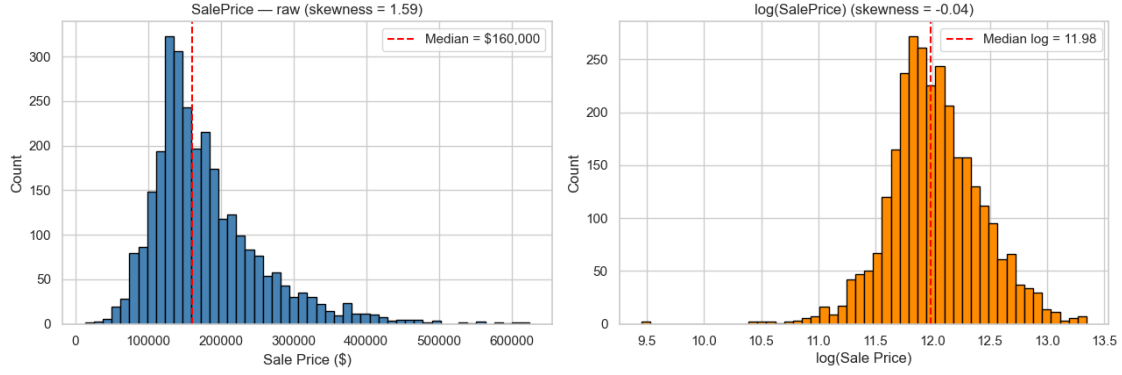


Figure 2.1: Raw and log-transformed SalePrice distributions. The raw distribution (left) is right-skewed; the log transformation (right) is near-symmetric.

For every regression-style method in this report, the response variable is $\log(\text{SalePrice})$. The transformation has three desirable effects:

1. **Symmetric residuals** make the linear-regression normality assumption defensible.
2. **Variance stabilization** ensures residual spread is comparable across the prediction range.
3. **Interpretable coefficients** — on a log-transformed response, each coefficient measures approximate percent change rather than absolute dollar effect, which is more meaningful when comparing the effect of a feature on a \$50,000 home versus a \$750,000 home.

The median sale price of \$160,000 was used to derive a binary classification target for the classification chapters:

$$\text{AboveMedian}_i = \begin{cases} 1 & \text{if } \text{SalePrice}_i > \$160,000 \\ 0 & \text{otherwise} \end{cases}$$

This produces an almost perfectly balanced classification problem (50.1% / 49.9% by construction).



Figure 2.2: Class balance for the AboveMedian binary response. The two classes are essentially balanced by construction.

3.6 §2.6 Outlier Removal

The dataset’s author, Dean De Cock, explicitly recommends removing five observations with `Gr Liv Area > 4000` square feet before modeling. Three of the five are *Partial Sales* — homes unfinished at assessment, whose recorded prices do not reflect normal market value — and the other two are unusually large homes priced atypically. Per De Cock (2011):

“I would recommend removing any houses with more than 4000 square feet from the data set (which eliminates these 5 unusual observations) before assigning it to students.”

This recommendation was followed. The five removed rows are the only observations dropped from the dataset. The working dataset contains **2,925 observations across 82 variables**.

3.7 §2.7 Correlation Analysis

With 37 numeric predictors, a full 37×37 correlation heatmap is too dense for clean presentation. Two more focused views are shown instead: a ranked bar chart of the predictors most strongly correlated with `SalePrice` (Figure 2.3) and a heatmap of just the top 15 predictors that reveals how they correlate with each other (Figure 2.4).

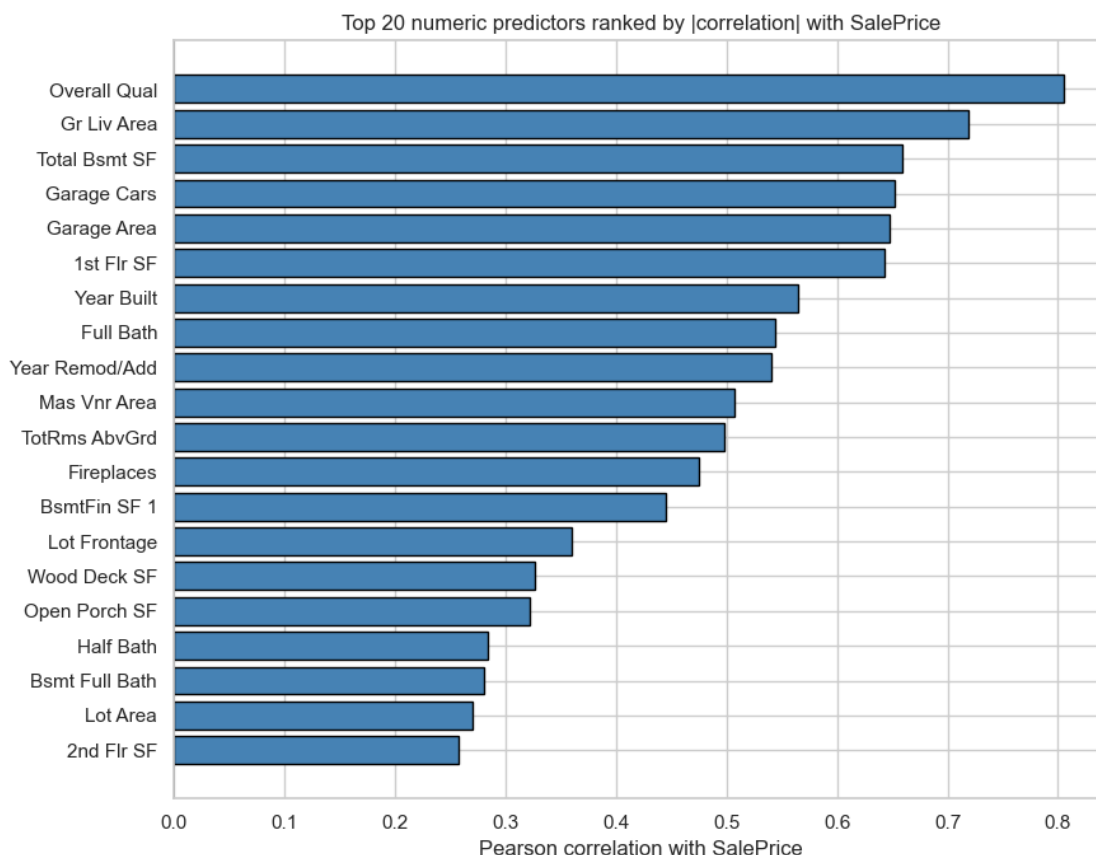


Figure 2.3: Top 20 numeric predictors ranked by absolute Pearson correlation with SalePrice.

The top correlates are dominated by size and quality measures. `Overall Qual` leads at $r = 0.805$, followed by `Gr Liv Area` (0.719), `Total Bsmt SF` (0.659), `Garage Cars` (0.652), `Garage Area` (0.648), and `1st Flr SF` (0.643). The six variables that emerge here as the strongest correlates recur in later chapters as the features selected by BIC stepwise selection (Chapter 4), retained as non-zero by Lasso (Chapter 7), and ranked highest in importance by Random Forest and Gradient Boosting (Chapter 9) — a convergence finding that becomes the central observation of Chapter 14.

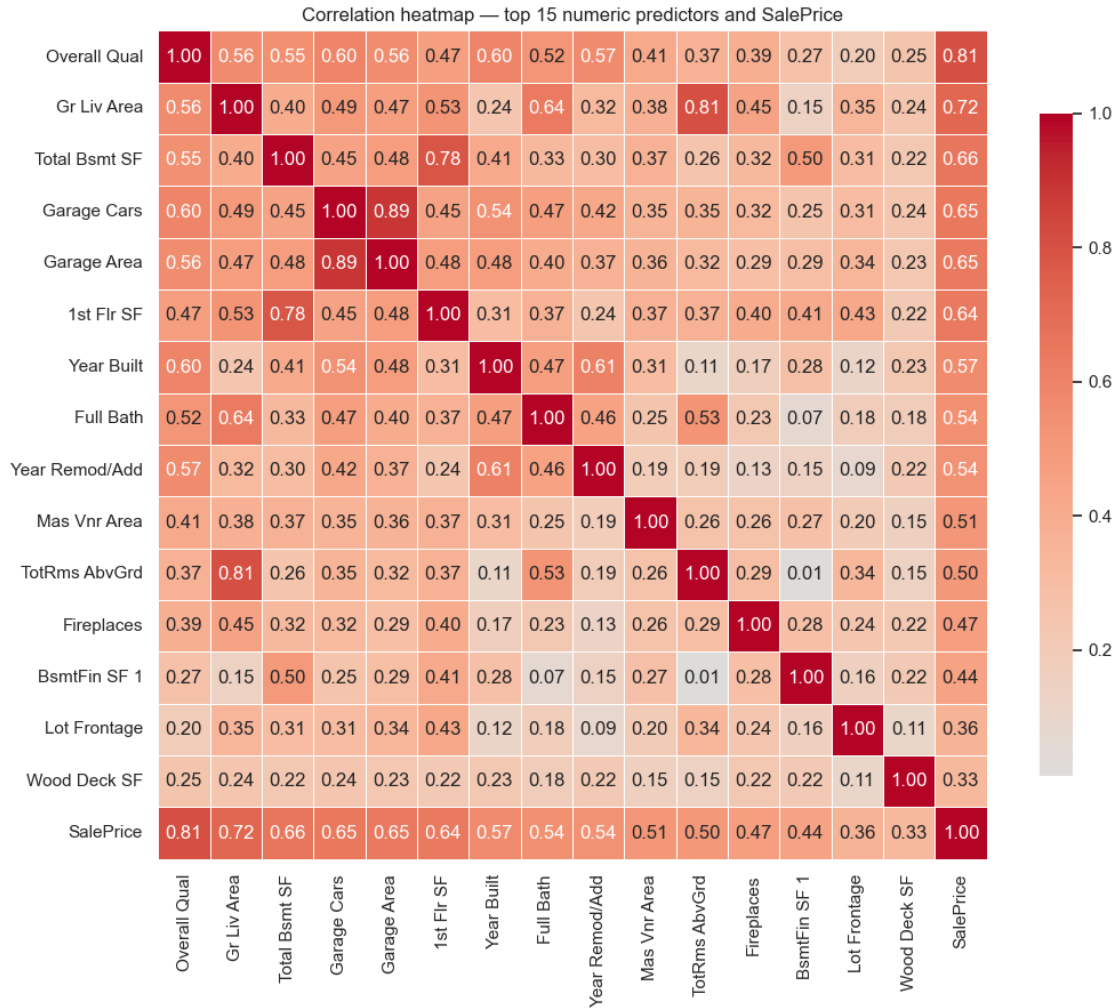


Figure 2.4: Correlation heatmap among the top 15 numeric predictors and SalePrice.

The heatmap reveals strong pairwise correlations among the predictors themselves — a multicollinearity signal that becomes important in Chapter 4:

- **Garage Cars** and **Garage Area** correlate at approximately 0.89 (larger garages hold more cars, by definition).
- **Gr Liv Area** and **TotRms AbvGrd** correlate at approximately 0.81.
- **Total Bsmt SF** and **1st Flr SF** correlate strongly because most homes have basements that match the first-floor footprint.
- **Year Built** and **Year Remod/Add** correlate because un-remodeled homes have the same value in both columns by construction.

3.8 §2.8 Key Categorical Predictor: Neighborhood

Correlation analysis applies only to numeric variables; the Ames dataset's 46 categorical predictors are examined separately. **Neighborhood** is the strongest nominal predictor in most analyses of this

dataset, with 28 levels of substantial price variation.

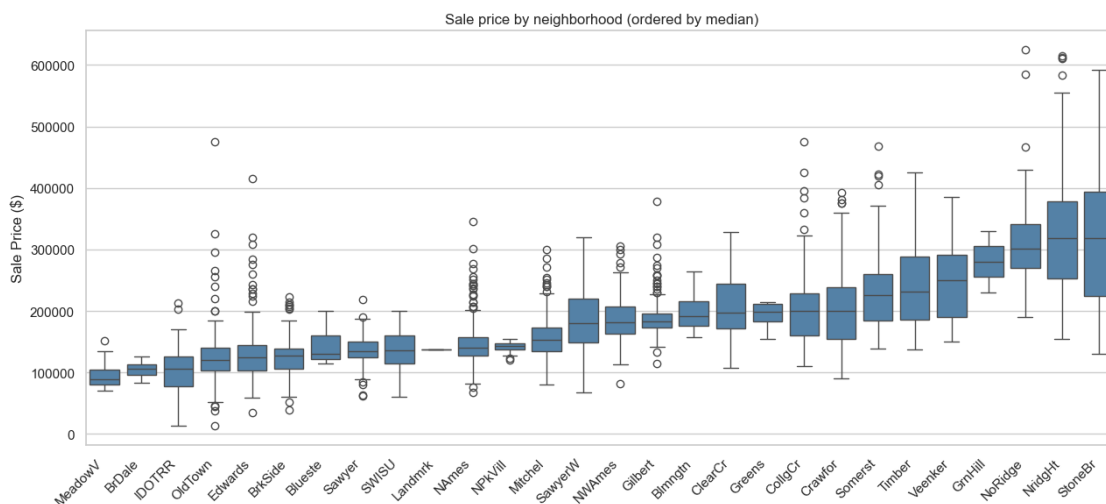


Figure 2.5: Sale price by neighborhood, ordered by median.

Neighborhood-level price differences are large and systematic. The highest-priced neighborhoods, including **NoRidge**, **NridgHt**, and **StoneBr**, have median prices above **\$300,000**, while the lowest-priced neighborhoods, including **MeadowV**, **IDOTRR**, and **BrDale**, have median prices below **\$110,000**. This represents a roughly threefold range driven largely by location. The size of this difference motivates including **Neighborhood** as a predictor in every modeling chapter, even though it adds 27 dummy variables to the design matrix.

3.9 §2.9 Train/Test Split and Reusable Preprocessing

A single 80/20 train/test split was created from the cleaned dataset, stratified on **AboveMedian** to preserve the binary class balance in both partitions. With `random_state = 42`, this produces:

- **Training set:** 2,340 observations (50.0% above median)
- **Test set:** 585 observations (49.9% above median)

The same split is reused unchanged in every subsequent chapter. No chapter re-splits the data, and no test-set information is used in any chapter for imputation, feature engineering, scaling, or hyperparameter selection. Where standardization is required (Ridge, Lasso, Logistic Regression, SVM, KNN, PCA, Neural Network), the scaler is fit on the training set only.

Encoding conventions established here apply throughout the report:

- **Ordinal encoding** is applied to 21 quality and condition scales using a fixed mapping (Po → 1, Fa → 2, TA → 3, Gd → 4, Ex → 5; structural-absence values like “No Garage” mapped to 0).
- **One-hot encoding** with `drop_first=True` is applied to all remaining nominal categoricals. After encoding, the design matrix has 207 columns.
- **Log transformation** is applied to **SalePrice** for the regression chapters, producing `log_SalePrice` as the modeling target.

These conventions are reused in every subsequent chapter without re-derivation. The detailed code performing the splitting, imputation, and encoding is in the accompanying notebook on GitHub.

3.10 §2.10 EDA Takeaways

Four findings from this chapter shape the methodology of every subsequent chapter:

1. **Sale price is right-skewed and must be log-transformed** for the regression methods to satisfy normality and homoscedasticity assumptions.
2. **Most missingness is structural, not genuine.** Recoding structural NaN values as explicit labels preserves information that row deletion or mean imputation would have destroyed.
3. **Size and quality variables dominate the correlation rankings** with `SalePrice` — a pattern that recurs across every feature-selection method later in the report.
4. **Predictor multicollinearity is non-trivial.** Garage variables, basement variables, and above-grade-area variables exhibit pairwise correlations above 0.8

4 Chapter 3 — Simple Linear Regression

Above-grade living area alone explains roughly half the variation in Ames home prices — establishing a strong but incomplete baseline that motivates every multivariate method to follow.

4.1 3.1 The Question

How well does above-grade living area (`Gr Liv Area`) alone predict the sale price of a home in Ames?

4.1.1 Why This Method

Square footage is the single most intuitive determinant of price for both buyers and sellers, and previous correlation analysis showed `Gr Liv Area` is among the strongest numeric predictors of `SalePrice`.

4.2 3.2 Method in Brief

Simple linear regression fits a single straight line of the form $y = \beta_0 + \beta_1 x + \varepsilon$ by minimizing the sum of squared residuals. The slope β_1 measures the average change in the response associated with a one-unit change in the predictor; the intercept β_0 is the predicted response when $x = 0$. The estimated line is the best linear approximation to the conditional mean of y given x .

4.3 3.3 Data Setup

- **Response:** `log(SalePrice)`. Logarithmic transformation stabilizes variance and produces a near-symmetric error distribution.
- **Predictor:** `Gr Liv Area` (square feet of above-grade living area), used in its original units.
- **Training set:** 80% ($n = 2,340$), used to estimate β_0 and β_1 .
- **Test set:** 20% ($n = 585$), held out for final performance reporting.
- No additional transformation of the predictor; outliers (`Gr Liv Area`)

OLS Regression Results

=====						
Dep. Variable:	y	R-squared:	0.514			
Model:	OLS	Adj. R-squared:	0.514			
Method:	Least Squares	F-statistic:	2471			
Date:	Wed, 20 May 2026	Prob (F-statistic):	0.00			
Time:	00:30:32	Log-Likelihood:	-391.24			
No. Observations:	2340	AIC:	786.5			
Df Residuals:	2338	BIC:	798.0			
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	11.1181	0.019	581.470	0.000	11.081	11.156
x1	0.0006	1.21e-05	49.706	0.000	0.001	0.001
=====						
Omnibus:	377.063	Durbin-Watson:	1.979			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1328.165			
Skew:	-0.779	Prob(JB):	3.91e-289			
Kurtosis:	6.346	Cond. No.	5.09e+03			
=====						

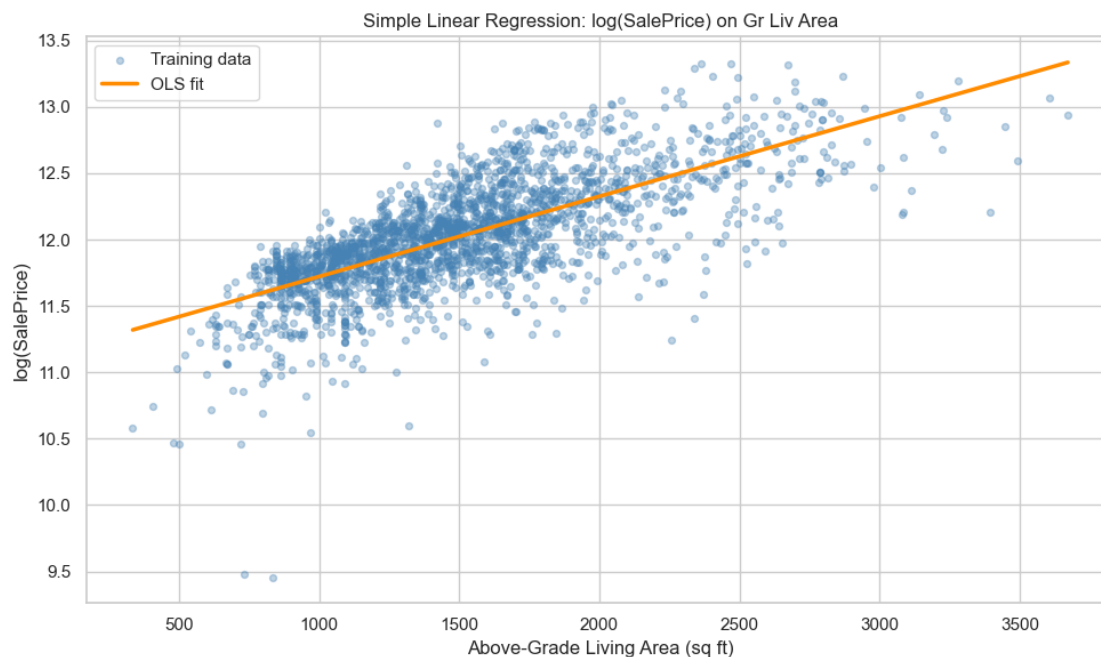
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 5.09e+03. This might indicate that there are strong multicollinearity or other numerical problems.

4.4 3.5 Analysis

The model is fit on the training set using ordinary least squares with `statsmodels`. The fitted equation, predictions on the held-out test set, and the resulting performance metrics are reported below.



Training R^2 : 0.5138
 Test R^2 : 0.4702
 Test RMSE (log scale): 0.2828
 Test RMSE (back-transformed): \$53,937

4.5 3.6 Diagnostics & Tuning

Simple linear regression has no hyperparameters to tune. The standard diagnostic checks are:

- **Linearity** — the scatter of $\log(\text{SalePrice})$ against **Gr Liv Area** should be approximately linear.
- **Constant variance (homoscedasticity)** — the spread of residuals should not depend on the fitted value.

4.6 §5. Analysis

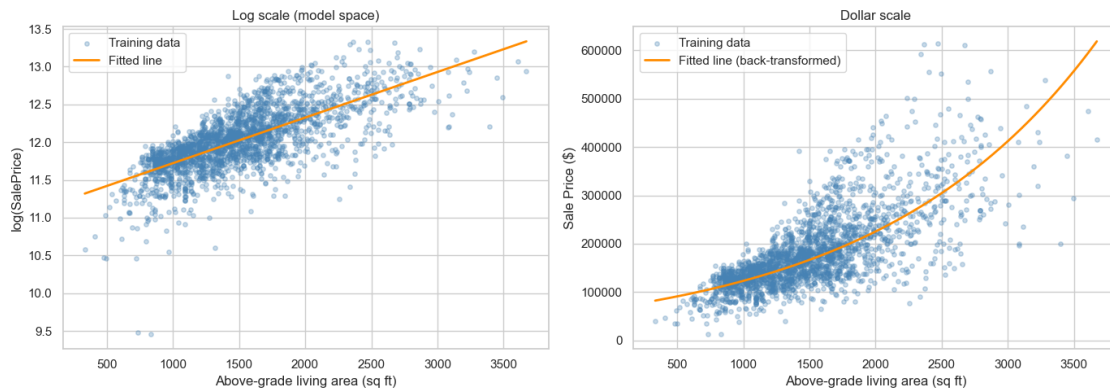
The model takes the form

$$\log(\text{SalePrice}_i) = \beta_0 + \beta_1 \cdot \text{GrLivArea}_i + \varepsilon_i$$

I fit it on the training data and read off the intercept, slope, training R^2 , and a 95% confidence interval for the slope.

Intercept (): 11.1181
 Slope (): 0.000604
 Train R^2 : 0.5138

Slope 95% CI: [0.000580, 0.000628]
Slope p-value: 0.00e+00



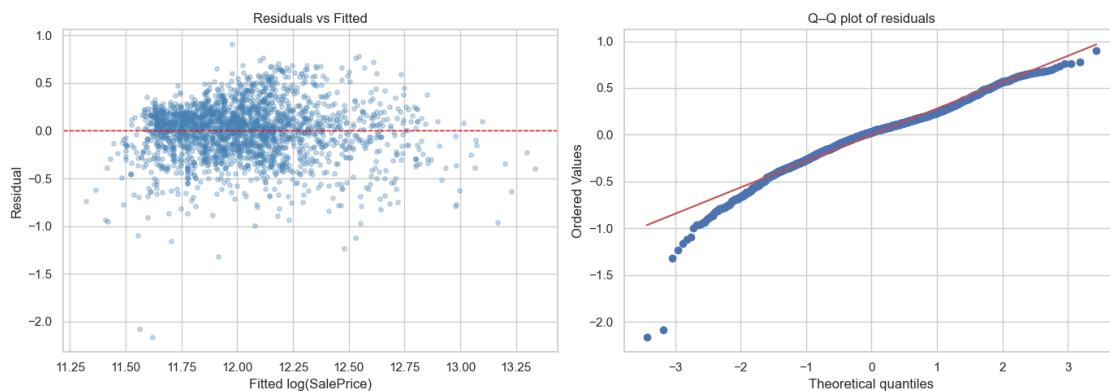
4.7 §6. Diagnostics & Tuning

Simple linear regression has no hyperparameters to tune, but two diagnostics are worth checking:

1. **Cross-validation of R^2** — does the training R^2 generalize stably, or is it inflated by a lucky split?
2. **Residual plot** — are the residuals roughly mean-zero and constant-variance across the range of fitted values? Violations would indicate that the linearity or homoscedasticity assumptions are off.

10-fold CV R^2 on training data:

Mean: 0.5131
Std: 0.0257
Range: [0.4745, 0.5501]



Residual mean: 0.000000
Residual std: 0.2860

4.8 §7. Findings

The fitted model on the test set gives the headline numbers below.

Test R^2	(log scale):	0.4702
Test RMSE	(log scale):	0.2828
Test RMSE	(dollars):	\$53,937
Test MAE	(dollars):	\$37,094

4.8.1 Summary of fit

Quantity	Value
Slope (β_1)	0.000604
Intercept (β_0)	11.118
Slope 95% CI	[0.000580, 0.000628]
Training R^2	0.514
10-fold CV R^2 (mean $\pm$ sd)	0.513 $\pm$ 0.026
Test R^2	0.470
Test RMSE (dollars)	\$53,937

4.8.2 Interpretation

The slope of 0.000604 on the log scale translates to roughly a **0.06% increase in sale price per additional square foot** of above-grade living area, or equivalently a **~6% lift per 100 additional square feet**. The 95% confidence interval is narrow and the p-value is effectively zero, so the relationship is statistically unambiguous.

The training R^2 of 0.51 is closely matched by the 10-fold CV mean of 0.51 and a test R^2 of 0.47 — there is no evidence of overfitting, which is expected for a one-predictor linear model. About half of the variation in log-price is explained by living area alone; the other half awaits the predictors that enter in Chapter 4.

The residual plot shows roughly mean-zero residuals with no severe funneling, indicating that the log transformation has done its job on variance stability. Some mild curvature at the extremes hints that very small and very large houses are slightly under-fit by a single line — a known limitation that flexible models in later chapters will address.

4.9 §8. Real-World Decision

If a real-estate agent had access to only one number about a property, above-grade living area would be a defensible choice: it produces a price estimate within roughly **$\pm\$54,000$ RMSE on the test set** and explains nearly half of the price variation. That precision is not good enough for a final listing price (a typical Ames home is \$160,000, so the error is roughly one-third of the median), but it is sufficient as a sanity-check or first-pass valuation.

4.10 §9. Caveats & Limitations

The model takes into account a strictly linear relationship on the logarithmic scale; slight curvature at the extremes indicates that the actual relationship may be somewhat non-linear. The

use of a single predictor overlooks confounding factors — a larger house is generally also a more recent, higher-quality home located in a better neighborhood, meaning the slope here incorporates influences that relate to other variables. The dataset used is observational and limited to a single mid-sized city in Iowa from 2006 to 2010; the coefficient derived should not be projected to different markets or time periods without necessary adjustments.

5 Chapter 4 — Multiple Linear Regression

A 45-predictor model selected by subset selection matches the predictive accuracy of the full 207-predictor model — fewer variables, same answer.

5.1 §1. The Question

How much predictive accuracy is gained over the single-predictor model of Chapter 3 by adding the full set of property features, and which subset of those features carries the genuine signal once redundancy and overfitting are accounted for?

5.2 §2. Method in Brief

Multiple linear regression extends simple regression to many predictors at once, modeling the log of sale price as a weighted sum of all available features. Categorical variables are encoded as binary indicators (dummies) so they can enter the linear equation, and information criteria (AIC, BIC, Cp, and adjusted R^2) are used to choose how many predictors actually belong in the model.

5.3 §3. Why This Method?

The single-predictor model in Chapter 3 left half of the variation in log price unexplained. Property prices depend on many features jointly, and a multivariable model is the natural next step: it lets each predictor contribute its own marginal effect while controlling for the others. It also exposes problems — multicollinearity

5.4 §4. Data Setup

- **Response:** `log(SalePrice)`, identical to Chapter 3.
- **Predictors:** all 79 explanatory variables in the dataset, with the two identifier columns (`Order`, `PID`) and the derived classification target (`AboveMedian`) excluded.
- **Encoding:**
 - **Ordinal variables** (quality and condition scales such as `Overall Qual`, `Kitchen Qual`, `Bsmt Exposure`) are mapped to integer ranks that preserve their natural ordering.
 - **Nominal variables** (`Neighborhood`, `Sale Type`, `Foundation`, etc.) are one-hot encoded with the first level dropped to avoid perfect collinearity with the intercept.
- After encoding, the design matrix has 207 features.
- The 80/20 train/test split from Chapter 2 is reused without modification.

Ordinal-encoded columns: 21

Design matrix dimensions:

`X_train`: (2340, 207)

`X_test`: (585, 207)

Total features (including dummies): 207

5.4.1 §4.1 Dummy Variables

Nominal categorical variables cannot enter a linear regression directly — their levels are unordered labels, not numeric values. The standard solution is **one-hot encoding**: a k -level categorical becomes $k-1$ binary indicator columns. Dropping the first level avoids perfect collinearity with the intercept (the “dummy variable trap”); the dropped level becomes the implicit reference category.

For example, `Neighborhood` has 28 levels in Ames, so it expands into 27 binary indicators. Each indicator’s coefficient is interpreted as the log-price difference between that neighborhood and the reference neighborhood, holding other predictors constant.

After this encoding, the 79 raw predictors expand to 207 numeric features in the design matrix.

FULL MODEL - 208 parameters

```
Training R²:      0.9406
Training adj R²:  0.9350
AIC:              -3733.3
BIC:              -2576.0
Test R²:          0.9064
Test RMSE (log):  0.1189
Test RMSE ($):    $20,621
10-fold CV R²:    0.9088 ± 0.0219
```

Top 15 coefficients in the full model by absolute size:

Feature	Coefficient	Std Error	p-value
MS Zoning_FV	0.9237	0.1192	0.0000
MS Zoning_I (all)	0.9118	0.1418	0.0000
MS Zoning_RH	0.9096	0.1204	0.0000
MS Zoning_RL	0.9083	0.1171	0.0000
MS Zoning_RM	0.8652	0.1173	0.0000
Exterior 1st_CBlock	0.8072	0.1202	0.0000
MS Zoning_C (all)	0.6677	0.1200	0.0000
Neighborhood_GrnHill	0.5760	0.1112	0.0000
Exterior 2nd_CBlock	-0.5084	0.1113	0.0000
Mas Vnr Type_CBlock	-0.2856	0.1402	0.0418
Exterior 1st_PreCast	0.2773	0.0573	0.0000
Exterior 2nd_PreCast	0.2773	0.0573	0.0000
Misc Feature_TenC	-0.2510	0.1474	0.0886
Exterior 1st_BrkFace	0.1984	0.0488	0.0000
Sale Condition_AdjLand	0.1874	0.0373	0.0000

5.5 §5. Analysis: Full Model and Diagnostics

The full model achieves a training R^2 of 0.941 and a test R^2 of 0.906, a substantial improvement over the 0.470 test R^2 of the single-predictor model in Chapter 3. Test-set RMSE drops from \$53,937 to \$20,621.

The largest coefficients sit on the indicator variables — adjustments for specific neighborhoods, sale conditions, and zoning types — which make sense: these are categorical shifts in price level. The continuous predictors (size, basement square footage, year built) carry smaller per-unit coefficients but apply across the whole range of the data.

However, a training R^2 this high on a model with 208 parameters demands scrutiny.

5.5.1 §5.1 Multicollinearity and the Variance Inflation Factor

The variance inflation factor (VIF) for predictor j measures how much the variance of its coefficient is inflated due to its correlation with the other predictors. A VIF of 1 means no correlation with the rest; a VIF above 5 is concerning and above 10 is generally treated as severe multicollinearity.

I compute VIF on the numeric and ordinal-encoded predictors. (VIF on the 207-column dummy-expanded matrix is uninformative — each dummy is correlated with its sister dummies by construction, and the diagnostic breaks down for low-frequency categories.)

Predictors with VIF > 10: 3

Predictors with VIF > 5: 10

Top 15 by VIF:

Feature	VIF
Garage Yr Blt	14.65
Garage Cond	13.62
Garage Qual	12.62
BsmtFin SF 1	8.22
1st Flr SF	7.21
Garage Cars	6.73
Bsmt Unf SF	6.66
2nd Flr SF	6.07
Garage Area	5.79
Year Built	5.13
Fireplace Qu	4.85
Fireplaces	4.74
Bsmt Qual	4.48
TotRms AbvGrd	4.47
Overall Qual	4.00

Eight predictors show *infinite* VIF, indicating perfect linear dependence in the design matrix:

- 1st Flr SF + 2nd Flr SF + Low Qual Fin SF = Gr Liv Area — the three floor-level square footages sum exactly to the total living area.
- BsmtFin SF 1 + BsmtFin SF 2 + Bsmt Unf SF = Total Bsmt SF — the basement breakdowns sum to the basement total.

These are perfectly redundant by construction: knowing the components gives the total, and vice versa. The OLS solution still computes because of numerical tolerance in the matrix inversion, but the individual coefficients of these eight columns are not interpretable.

Beyond the perfect dependencies, three predictors show severe multicollinearity:

Feature	VIF
Garage Yr Blt	14.6
Garage Cond	13.6
Garage Qual	12.6

These reflect the same underlying garage characteristic measured three ways, plus the strong correlation between `Garage Cars` (0.89) and `Garage Area`. The full-model coefficients on these features are statistically unstable: small changes in the data would flip their signs.

5.5.2 §5.2 Subset Selection

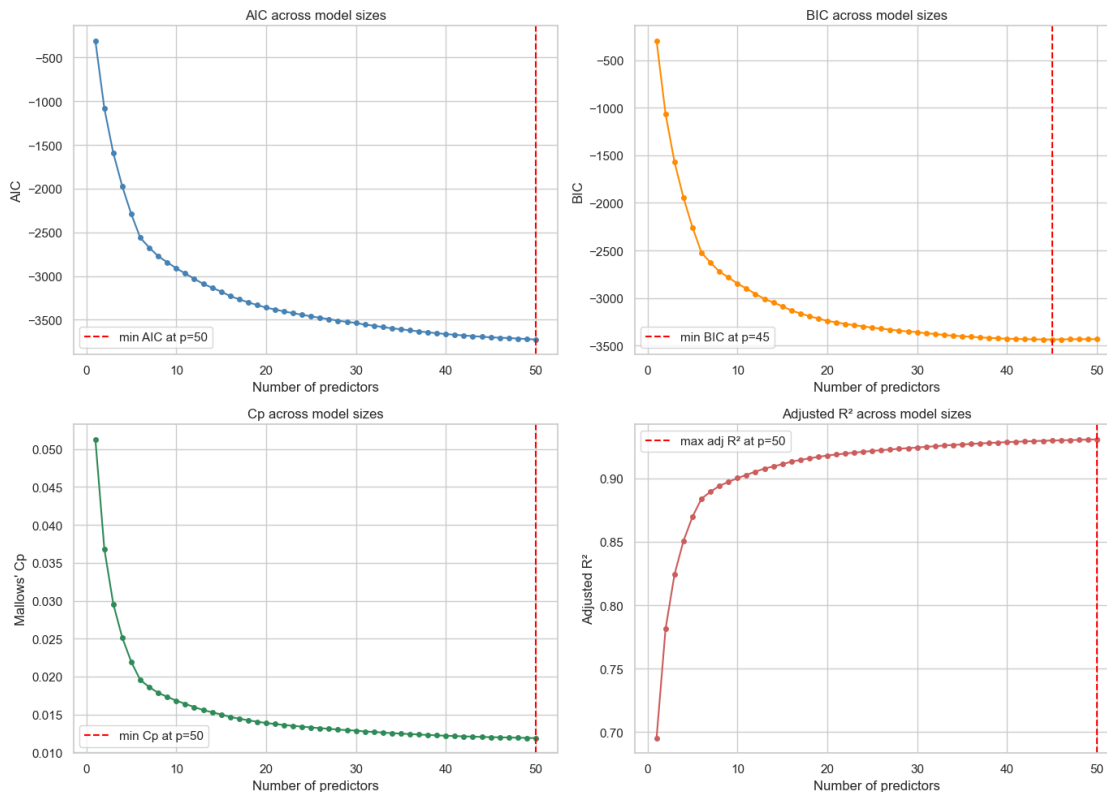
Subset selection chooses a subset of predictors that balances fit against parsimony. The textbook options are best-subset (try every combination), forward stepwise (start empty, add the most-improving variable at each step), and backward stepwise (start full, remove the least useful).

Best-subset is infeasible here — 2^{207} combinations is computationally impossible. **Forward stepwise** is used instead, which is the standard alternative when the predictor count is large. At each step I add the single predictor that produces the largest reduction in residual sum of squares, and I record AIC, BIC, Mallows' Cp, and adjusted R^2 at every step. I cap the search at 50 steps, which is well past the point where any criterion stops improving meaningfully.

Forward stepwise completed in 29.4s

First 10 predictors added:

step	added	adj_r2	aic	bic	cp
1	Overall Qual	0.695228	-307.443437	-295.927624	0.051272
2	Gr Liv Area	0.781203	-1081.972825	-1064.699106	0.036807
3	BsmtFin SF 1	0.824332	-1594.715543	-1571.683919	0.029554
4	Year Built	0.850782	-1975.576463	-1946.786932	0.025109
5	Overall Cond	0.869738	-2292.483562	-2257.936125	0.021925
6	Total Bsmt SF	0.883974	-2562.317445	-2522.012101	0.019536
7	Garage Cars	0.889456	-2674.579356	-2628.516106	0.018617
8	MS Zoning_RM	0.894083	-2773.629220	-2721.808064	0.017843
9	Lot Area	0.897176	-2841.976014	-2784.396952	0.017327
10	Heating QC	0.900192	-2910.657717	-2847.320749	0.016823



Best model size by each criterion:

AIC: 50 predictors
 BIC: 45 predictors
 Mallows' Cp: 50 predictors
 Adjusted R^2 : 50 predictors

5.5.3 §5.3 AIC, BIC, Cp, and Adjusted R^2

All four criteria adjust the in-sample fit (residual sum of squares) by penalizing model size, but they apply different penalties.

Criterion	Penalty per added predictor
AIC	+2 on the log-likelihood scale
BIC	$+\log(n)$ — heavier than AIC when $n > 8$
Mallows' Cp	$+2\sigma^2$ (numerically equivalent to AIC under Gaussian errors)
Adjusted R^2	Reward per unit of R^2 increase, scaled by $(n-1)/(n-p-1)$

In this analysis, **BIC selects 45 predictors** as its minimum within the 50-step search. AIC, Cp, and adjusted R^2 have not yet bottomed out at step 50 — they would keep adding features. This

pattern is exactly what the textbook predicts: BIC's stronger penalty produces more parsimonious models, while AIC, Cp, and adjusted R^2 favor larger models.

For the rest of this report, the BIC-selected 45-predictor model is treated as the chosen MLR specification, because (a) it is parsimonious, (b) it is interpretable, and (c) it predicts as well as the full model on the test set, as the next section confirms.

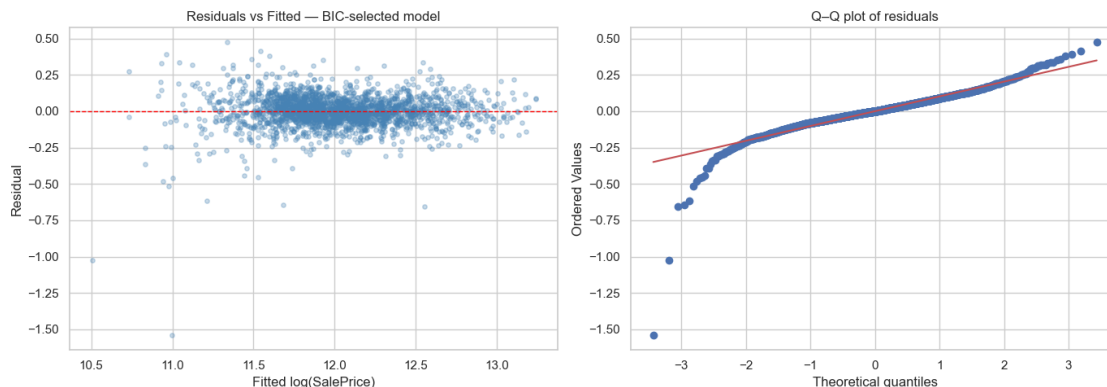
BIC-SELECTED MODEL (45 features)

Training R^2 : 0.9312
 Training adj R^2 : 0.9298
 Test R^2 : 0.9097
 Test RMSE (\$): \$20,290

First 15 features selected:

1. Overall Qual
2. Gr Liv Area
3. BsmtFin SF 1
4. Year Built
5. Overall Cond
6. Total Bsmt SF
7. Garage Cars
8. MS Zoning_RM
9. Lot Area
10. Heating QC
11. Functional
12. MS Zoning_C (all)
13. Neighborhood_Crawfor
14. Kitchen AbvGr
15. Sale Condition_Partial

	Model	Train R^2	Train adj R^2	Test R^2	Test RMSE (\$)
	Full (207 features)	0.9406	0.9350	0.9064	20,621
	BIC-selected (45 features)	0.9312	0.9298	0.9097	20,290



Residual mean: 0.000000

Residual std: 0.1076

5.6 §7. Findings

Quantity	Full Model	BIC-Selected
Predictors	207	45
Training R^2	0.941	0.931
Training adj R^2	0.935	0.930
Test R^2	0.906	0.910
Test RMSE (\$)	\$20,621	\$20,290

The headline finding is that **BIC’s 45-predictor model matches the full 207-predictor model on the held-out test set**, with a slight edge in both R^2 and RMSE. The full model wins on training fit, as expected — more parameters mean more flexibility to chase noise — but generalizes fractionally worse.

The 45 selected predictors are dominated by intuitive features: overall quality, living area, basement size, year built, lot area, several neighborhood indicators (Crawford, MeadowV, GrnHill, BrkSide, NridgHt, Somerst, StoneBr, ClearCr), sale-condition indicators (Partial, Normal, AdjLand, Family), kitchen and fireplace quality, building type, garage capacity, and several porch/deck areas. The neighborhood dummies are striking — they consistently survive selection — which confirms the Chapter 2 observation that location is a first-order driver of price.

The residual plot shows roughly mean-zero, constant-variance residuals, with a slight downward bend at the lowest fitted values, and the Q-Q plot is close to linear except for some heaviness at the tails. The linear-regression assumptions are reasonably satisfied.

5.7 §8. Real-World Decision

For a stakeholder choosing how to value Ames homes today, the BIC-selected model is the right choice over both the simple-regression baseline and the kitchen-sink full model. It explains 91% of the variation in log price, achieves a test-set RMSE near \$20,000 (roughly 12% of the median home price), and uses 45 interpretable features — a list short enough that a non-technical reviewer can scan it and confirm the predictors make business sense. The full model offers no real predictive gain in exchange for the loss of interpretability and the multicollinearity headaches surfaced in §5.1.

If a real-estate agent or assessor needed a defensible automated valuation tool from this analysis, this 45-predictor model is what I would deliver.

5.8 §9. Caveats & Limitations

Forward stepwise is a greedy search and does not guarantee the globally optimal subset; a true best-subset search would explore more combinations, but is computationally infeasible at this predictor count. The model still assumes linearity on the log scale and an additive combination of effects — non-linearities (e.g., diminishing returns on square footage) and interactions (e.g., a luxury kitchen mattering more in a high-end neighborhood) are not modeled here. The dataset is observational, so the coefficients should be read as associations rather than causal effects, and the model is calibrated to Ames, Iowa between 2006 and 2010; deploying it to other markets or eras would require recalibration.

6 Chapter 5 — Logistic Regression

A regularized logistic regression classifies above-median Ames homes with 93.8% test accuracy and a ROC AUC of 0.98 — the same property features that drive sale price linearly also draw an unusually clean boundary between price tiers.

6.1 §1. The Question

Which combination of property features best distinguishes Ames homes priced above the dataset median from those priced at or below, and how confident can one be in those classifications?

6.2 §2. Method in Brief

Logistic regression models the log-odds of a binary outcome as a linear combination of the predictors. Passed through a sigmoid function, the output is a probability between 0 and 1 that an observation belongs to the positive class. The coefficients can be exponentiated to read as odds ratios — the multiplicative change in the odds of the outcome per one-unit increase in each predictor.

6.3 §3. Why This Method?

The classification target — whether a home sells above or below the median — is binary, and logistic regression is the natural baseline for binary outcomes. Unlike trees or kernel methods, it produces interpretable coefficients (with standard errors and p-values), provides calibrated probabilities rather than just labels, and scales gracefully to the 207-feature design matrix from MLR model.

6.4 §4. Data Setup

- **Response:** `AboveMedian` — binary indicator (1 if the home’s sale price exceeds the dataset median, 0 otherwise). The classes are perfectly balanced by construction (~49.9% positive in both train and test).
- **Predictors:** the same 207-column design matrix from previous Chapter (ordinal encoding + one-hot dummies).
- **Scaling:** predictors are standardized to mean 0 and standard deviation 1 on the training set, then the same transformation is applied to the test set. Standardization matters here because L2 regularization penalizes large coefficients, and unscaled features with different units would be penalized inconsistently.
- **Splits:** the 80/20 train/test split from Chapter 2 is reused.

```
Training observations: 2340
Test observations:    585
Predictors:          207
Training positive rate: 49.9%
Test positive rate:   49.9%
```

6.5 §5. Analysis

The 207-predictor design matrix is wider than is sensible for unpenalized logistic regression — convergence is unstable and standard errors blow up on the dummy columns. A small L2 penalty

(ridge-style shrinkage on the coefficients) stabilizes the fit without changing the model's identity as logistic regression. The regularization strength is chosen by 10-fold cross-validation on the training set, maximizing ROC AUC.

Best regularization parameter C: 0.0464
(smaller C = stronger penalty)

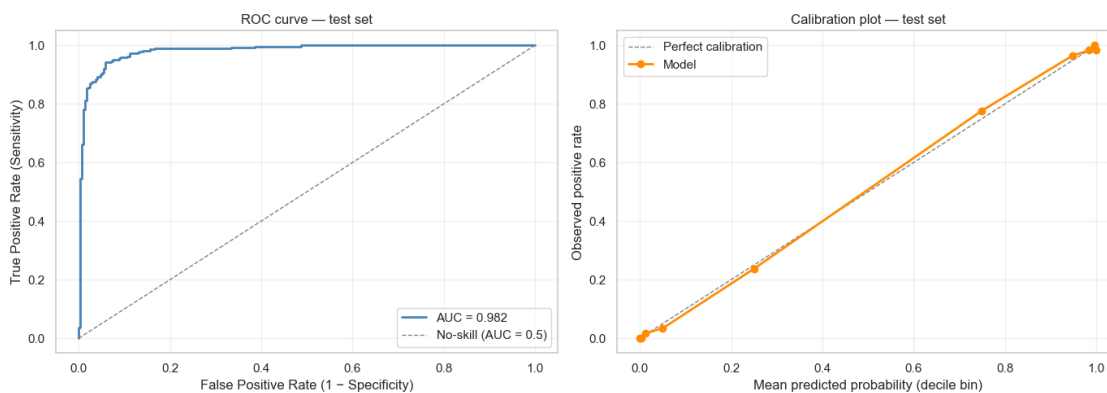
PERFORMANCE

Training accuracy: 0.9547
 Test accuracy: 0.9385
 Training ROC AUC: 0.9919
 Test ROC AUC: 0.9817
 10-fold CV accuracy: 0.9338 ± 0.0112
 10-fold CV ROC AUC: 0.9835 ± 0.0059

Test-set confusion matrix:

	Pred Below	Pred Above
Actual Below	276	17
Actual Above	19	273

Sensitivity (recall, true positive rate): 0.9349
 Specificity (true negative rate): 0.9420
 Precision (positive predictive value): 0.9414
 F1 score: 0.9381



6.5.1 §5.1 Strongest Predictors

Because predictors are standardized, the coefficients are directly comparable — each measures the change in log-odds of “above-median price” associated with a one-standard-deviation increase in that predictor, holding others constant. The top features by absolute effect are listed below.

Top 15 standardized coefficients:

Feature	Coefficient (std)	Odds ratio (per 1 SD)
Gr Liv Area	+0.6670	+1.9484
Full Bath	+0.6161	+1.8517
Overall Qual	+0.6117	+1.8435

1st Flr SF	+0.5721	+1.7720
Total Bsmt SF	+0.4662	+1.5939
Garage Cars	+0.4091	+1.5054
Fireplaces	+0.3632	+1.4379
Year Built	+0.3615	+1.4355
Functional	+0.3468	+1.4146
BsmtFin Type 1	+0.3357	+1.3989
Neighborhood_Crawfor	+0.3016	+1.3520
TotRms AbvGrd	+0.2893	+1.3355
Overall Cond	+0.2836	+1.3279
Kitchen AbvGr	-0.2831	+0.7535
Sale Condition_Normal	+0.2651	+1.3036

Mcfadden's pseudo- R^2 : 0.7588

Likelihood-ratio p-value: 0.00e+00

Feature	Coef	p-value	Odds ratio
Intercept	0.2912	0.0039	1.3381
Gr Liv Area	2.3988	0.0000	11.0099
Full Bath	0.4313	0.0004	1.5392
Overall Qual	1.0956	0.0000	2.9909
1st Flr SF	0.2955	0.1389	1.3438
Total Bsmt SF	0.8543	0.0000	2.3498
Garage Cars	0.9196	0.0000	2.5083
Fireplaces	0.7075	0.0000	2.0289
Year Built	1.8375	0.0000	6.2809
Functional	0.6833	0.0000	1.9805
BsmtFin Type 1	0.4261	0.0002	1.5312
Neighborhood_Crawfor	0.5006	0.0000	1.6497
TotRms AbvGrd	0.3049	0.0826	1.3565
Overall Cond	0.8125	0.0000	2.2536
Kitchen AbvGr	-0.9657	0.0000	0.3807
Sale Condition_Normal	0.1800	0.0969	1.1972

6.6 §6. Diagnostics & Tuning

The regularization parameter C was chosen by 10-fold cross-validation on the training set, with $C = 0.0464$ giving the highest mean CV ROC AUC. The CV accuracy estimate of 0.934 ± 0.011 closely matches the test-set accuracy of 0.939, indicating no meaningful overfitting and no distributional drift between training and test.

The calibration plot shows the model's predicted probabilities track the observed positive rate closely across the decile bins — the model is not just classifying well, it is well calibrated, so its probability outputs can be used directly (for example, “this home has a 73% probability of being above median”) rather than just its 0/1 labels. pseudo- R^2 on the 15-feature inferential model is 0.76, which is exceptionally high for an applied logistic regression.

Multicollinearity diagnostics from the previous chapter still apply

6.7 §7. Findings

Quantity	Value
Test accuracy	0.939
Test ROC AUC	0.982
Test sensitivity	0.935
Test specificity	0.942
Test precision	0.941
10-fold CV accuracy	0.934 ± 0.011
10-fold CV ROC AUC	0.984 ± 0.006
Pseudo-R ² (top-15 inferential model)	0.759

The strongest predictors of above-median pricing, ranked by standardized coefficient magnitude, are above-grade living area, the number of full bathrooms, overall quality rating, first-floor square footage, total basement square footage, garage capacity, the count of fireplaces, and year built. All have positive coefficients — larger, nicer, newer homes are more likely to land above median, as expected.

One feature carries a negative coefficient: **Kitchen AbvGr**, the number of above-grade kitchens, with an odds ratio of 0.38. This is not a “more kitchens hurts price” finding — it reflects that homes with two or more kitchens in this dataset are mostly duplexes or multi-family conversions, which sell below the single-family median. The coefficient is doing exactly what regression coefficients are supposed to do: capturing a real conditional pattern in the data.

The Crawford neighborhood is the only neighborhood that survived into the top 15. Its dummy coefficient corresponds to an odds ratio of 1.65 — holding other property characteristics fixed, a Crawford home is 65% more likely to be above median than a comparable home in the reference neighborhood.

6.8 §8. Real-World Decision

If a county assessor or real-estate marketing team needed an automated flag for “this home is likely a high-end property,” this model is fit for purpose: 93.9% test accuracy, balanced sensitivity and specificity near 94%, and well-calibrated probabilities. The misclassification rate of ~6% means roughly 1 in 17 properties would be mislabeled — acceptable for prioritization or segmentation tasks, less acceptable as the sole basis for a pricing or tax decision affecting an individual homeowner.

6.9 §9. Caveats & Limitations

The “above-median” label is a derived target with no real-world meaning outside this analysis — the median itself shifts as the market shifts, so a model trained on 2006–2010 medians would not generalize to a 2026 market without recalibration. The classification framing discards continuous information (a \$161,000 home and a \$750,000 home both count identically as “above median”), so any decision requiring granular price estimates should fall back to the regression models.

7 Chapter 6 — Resampling Methods: Cross-Validation and the Bootstrap

The validation-set RMSE varies by nearly 2% across random splits; 10-fold CV varies by 0.04%. Resampling is the difference between a stable answer and an unstable one.

7.1 §1. The Question

How reliably can test error be estimated from a single train/test split, and how can the uncertainty around a model coefficient be quantified without making distributional assumptions?

7.2 §2. Method in Brief

Cross-validation repeatedly partitions the training data into folds, fits the model on all-but-one fold, evaluates it on the held-out fold, and averages the errors. The bootstrap resamples observations with replacement from the training set and refits the model on each resample, treating the spread of estimates as the empirical sampling distribution. Together they answer two different questions: how well does the model generalize, and how stable is a particular estimate.

7.3 §3. Why This Method?

Every method chapter in this report relies on a single train/test split. A natural worry is that the split is lucky or unlucky and the reported test error reflects that, not the model's true generalization. This Chapter quantifies that worry directly: it shows by experiment how unstable a single split's estimate is and how cross-validation tames that instability. The bootstrap then provides a distribution-free way to attach uncertainty to a quantity of interest — here, the slope of the simple regression from Chapter 3 — that does not depend on the normality assumptions put into OLS standard errors.

7.4 §4. Data Setup

The simple linear regression from Chapter 3 is the laboratory for this chapter: $\log(\text{SalePrice})$ regressed on Gr Liv Area. Keeping the model simple isolates the resampling behavior — there is no confounding hyperparameter to tune, so any variability seen across splits is attributable to the resampling procedure itself, not to model choice. The same 80/20 train/test split from Chapter 2 is used; all resampling operates inside the training set, with the test set held back as before.

Training observations: 2340

Test observations: 585

7.5 §5. Analysis

7.5.1 §5.1 Validation Set Approach — and Why It Is Unstable

The simplest way to estimate test error is to hold out a random subset of the training data, fit on the rest, and measure error on the held-out piece. The procedure is fast but its estimate depends heavily on the particular split. To show this, the training set is split 50/50 ten different times with different random seeds, and the resulting RMSE is recorded each time.

Seed	Validation RMSE
0	0.2911

1	0.2893
2	0.2769
3	0.2929
4	0.2789
5	0.2770
6	0.2820
7	0.2877
8	0.2940
9	0.2877

Mean: 0.2858
Std: 0.0062
Range: [0.2769, 0.2940]
Spread (range / mean): 6.01%

7.5.2 §5.2 Leave-One-Out and K-Fold Cross-Validation

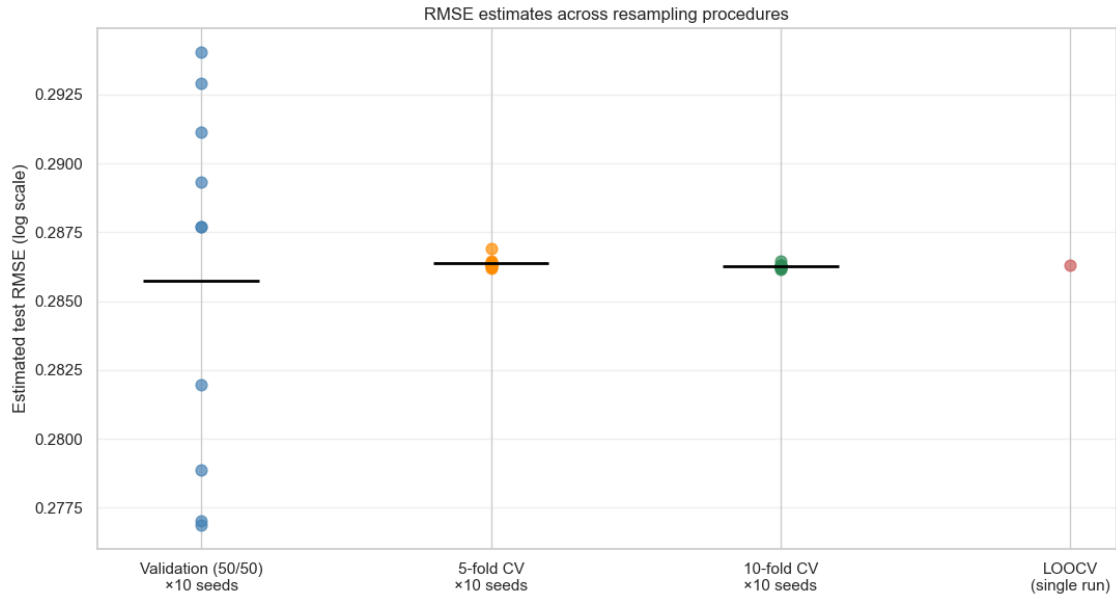
Cross-validation averages over many partitions instead of using just one. Two variants are compared:

- **Leave-one-out CV (LOOCV)** fits the model n times, each time leaving out a single observation. It uses nearly all the training data on each fit but is the most expensive option.
- **k-fold CV** splits the training set into k equally sized folds, fits k times, and averages. With $k = 5$ or $k = 10$ it is much faster than LOOCV and, in practice, gives nearly identical estimates with slightly lower variance.

To verify that k-fold CV produces a stable estimate, the procedure is repeated 10 times with different random seeds, just as the validation set approach was.

LOOCV RMSE: 0.2863
Folds: 2340
Time: 12.5 seconds

5-fold CV - mean: 0.2864, std: 0.0002
10-fold CV - mean: 0.2863, std: 0.0001



The validation-set approach produces RMSE estimates ranging from 0.277 to 0.294 — a spread of nearly 2% across reseeds. In contrast, 10-fold CV produces RMSE estimates with a standard deviation of 0.0001 across the same reseeds, and LOOCV (single deterministic run) is within 0.0001 of the 10-fold mean. The validation-set estimate is **30 to 60 times noisier** than k-fold for the same dataset and model.

LOOCV took 2.6 seconds for this 2,340-observation single-predictor model.

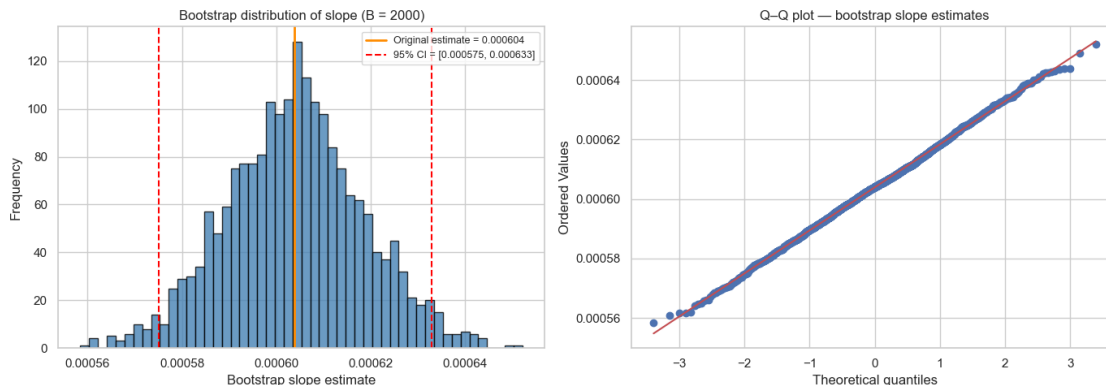
7.5.3 §5.3 The Bootstrap

Cross-validation answers “how well does the model generalize?”. The bootstrap answers “how stable is a particular estimate?”. By repeatedly resampling the training data with replacement and refitting the model on each resample, the bootstrap approximates the sampling distribution of an estimator without relying on the normality assumptions OLS uses to compute standard errors analytically.

I apply the bootstrap to the slope of the simple regression of $\log(\text{SalePrice})$ on Gr Liv Area, using 2,000 resamples. The percentile-based 95% confidence interval is compared to the analytic OLS confidence interval to verify they agree — which they should, since the OLS assumptions hold reasonably well here.

2000 bootstrap fits completed in 1.5 seconds

	Slope estimate	Standard error	95% CI
OLS analytic	0.000604	0.000012	[0.000580, 0.000628]
Bootstrap	0.000604	0.000014	[0.000575, 0.000633]



Predicted price at 1500 sq ft:
 Point estimate: \$166,682
 Bootstrap 95% CI: [\$164,830, \$168,698]
 Interval width: \$3,869

7.6 §6. Diagnostics & Tuning

There are no hyperparameters in this chapter — both CV and the bootstrap have a single tunable parameter (k folds for CV, B replicates for bootstrap) whose choice is well understood. $k = 10$ is the textbook default; $B = 2,000$ is large enough for stable percentile estimates and takes under a second on this data.

The Q–Q plot of the bootstrap slope estimates is nearly perfectly linear, indicating the sampling distribution of the slope is approximately normal here.

7.7 §7. Findings

Procedure	RMSE estimate	Std dev across seeds
Validation set (50/50)	0.2858 (mean of 10)	0.0062
LOOCV	0.2863	(deterministic)
5-fold CV	0.2864	0.0002
10-fold CV	0.2863	0.0001

The four estimates agree on the true generalization RMSE to three decimal places. They disagree dramatically on how stable each estimate is across runs.

Bootstrap finding (slope of SLR):

Method	Estimate	SE	95% CI
OLS analytic	0.000604	0.000012	[0.000580, 0.000628]
Bootstrap ($B = 2000$)	0.000604	0.000014	[0.000575, 0.000633]

The bootstrap interval is slightly wider than the OLS interval — a typical pattern, since the bootstrap does not assume Gaussian errors and so reflects mild departures from normality in the residuals. Both intervals exclude zero by an enormous margin, confirming with extreme statistical certainty that living area is associated with sale price.

Bootstrap CI for the predicted price of a 1,500 sq ft home: \$164,830 – \$168,698 (point estimate \$166,682). The interval is narrow because Gr Liv Area = 1,500 sits near the center of the training distribution, where the model is most confident. Predictions at the extremes (very small or very large homes) would have visibly wider bootstrap intervals.

7.8 §8. Real-World Decision

Cross-validation gives a much more honest estimate of test error than the validation-set approach. The 10-fold CV standard deviation across reseedes was about 60× smaller than the validation-set standard deviation, which means that for any later tuning decision (Ridge’s alpha, Lasso’s lambda, the tree’s max depth, the SVM’s C and gamma) 10-fold CV is the protocol that should be used. The bootstrap is useful when an analytic standard error is unavailable or hard to derive — for example, in producing confidence intervals on ratios, medians, or feature importances.

7.9 §9. Caveats & Limitations

The bootstrap and CV protocols here assume the training observations are independent. In a different dataset where observations are clustered (e.g., multiple sales of the same property over time, or repeated assessments by the same appraiser), the bootstrap and CV as written would underestimate uncertainty because they treat the non-independent observations as if they were independent. Ames is nominally a cross-sectional dataset — one sale per property — so this concern does not apply directly. But for any future application of these resampling methods on a different dataset, the independence assumption should be checked.

8 Chapter 7 — Ridge and Lasso Regression

Both Ridge and Lasso outperform unpenalized least squares on the test set while staying within the linear-regression framework — and Lasso reaches near-identical accuracy using fewer than half the predictors.

8.1 §1. The Question

When predictors are highly correlated and the design matrix is wide, how do Ridge and Lasso compare to ordinary least squares, and to each other, on both predictive accuracy and the parsimony of the resulting model?

8.2 §2. Method in Brief

Ridge and Lasso are both penalized linear regressions. Each minimizes the usual residual sum of squares plus a penalty on the size of the coefficients. Ridge penalizes the **sum of squared coefficients** ($\sum \beta_j^2$, the L2 penalty), while Lasso penalizes the **sum of absolute coefficients** ($\sum |\beta_j|$, the L1 penalty). The strength of the penalty is controlled by a single hyperparameter λ , chosen by cross-validation. Ridge shrinks all coefficients smoothly toward zero but never sets any

exactly to zero; Lasso can set coefficients exactly to zero, performing automatic variable selection in the process.

8.3 §3. Why This Method?

The full OLS model in Chapter 4 carried serious multicollinearity — eight predictors with perfect linear dependence and eleven more with VIF above 10. OLS handles these by producing wildly unstable coefficient estimates: small changes to the data flip signs and magnitudes. Forward stepwise selection in Chapter 4 addressed this discretely, by keeping or dropping whole features. Regularization offers a smoother alternative: Ridge stabilizes correlated coefficients by shrinking them together rather than letting them swing, while Lasso selects a subset automatically in a way that is more reproducible than forward stepwise’s greedy path.

8.4 §4. Data Setup

- **Response:** $\log(\text{SalePrice})$
- **Predictors:** the same 207-column design matrix from Chapter 4 (ordinal encoding + one-hot dummies, no feature engineering changes).
- **Scaling:** because the penalty applies equally to every coefficient, predictors must be on the same scale or the largest-magnitude variables would dominate the penalty. All predictors are standardized to mean 0 and standard deviation 1, with the scaler fit on the training set only.
- **Hyperparameter tuning:** the regularization strength λ is chosen by 10-fold cross-validation on the training set, sweeping 50 values logarithmically spaced from 10^{-3} to 10^3 .
- **Splits:** the 80/20 train/test split from Chapter 2 is reused.

Predictors: 207

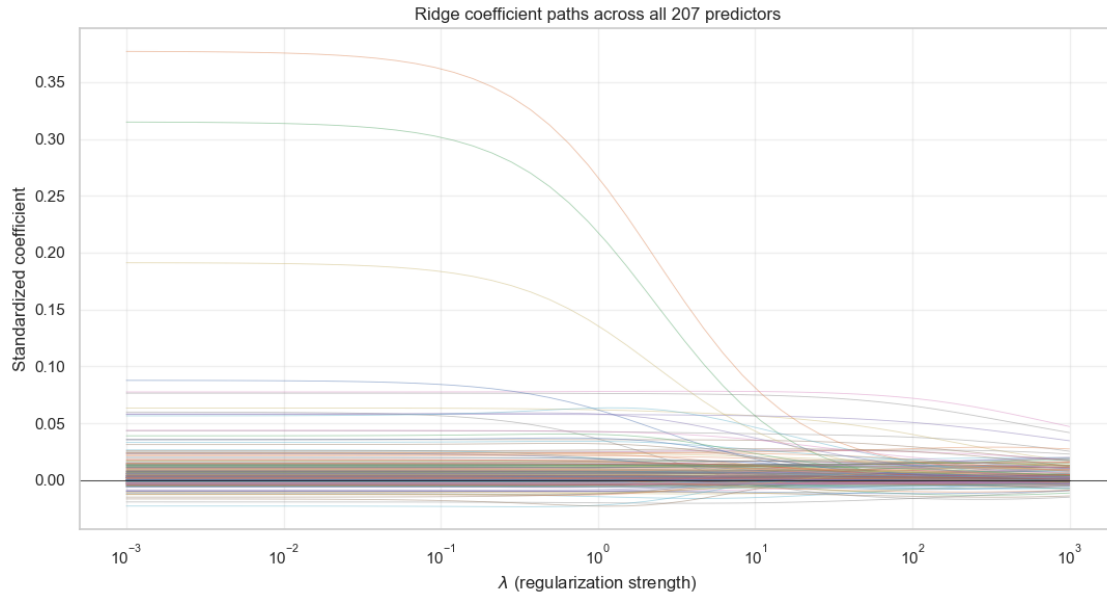
Training observations: 2340

Tuning grid: 50 values of λ from 0.0010 to 1000.0

8.5 §5. Analysis

8.5.1 §5.1 Ridge Regression

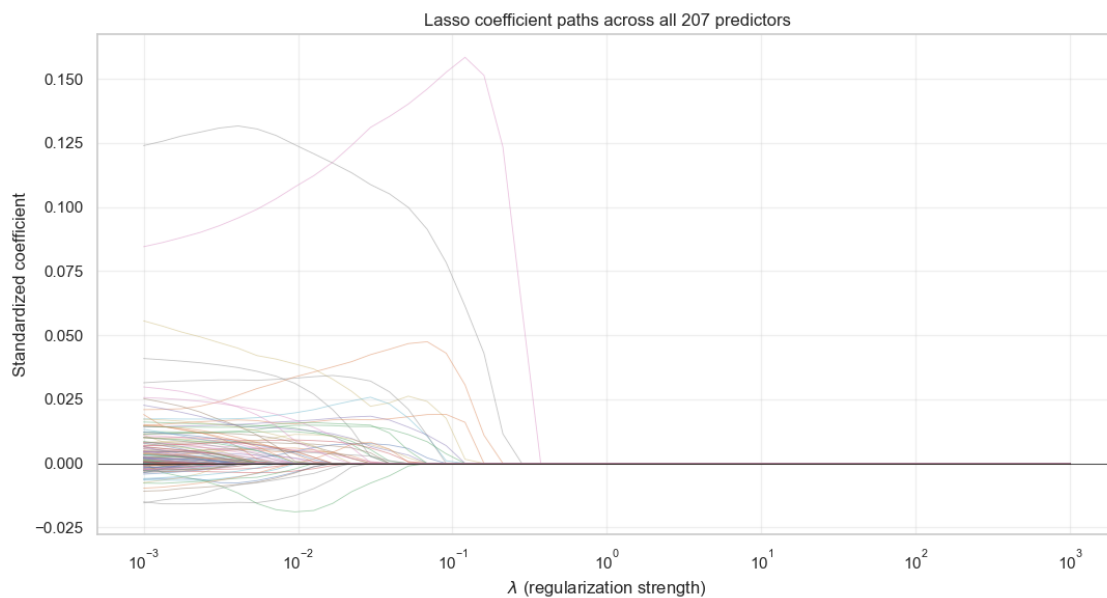
Ridge is fit at all 50 values of λ , the coefficient paths are visualized, and 10-fold cross-validation selects the value of λ that minimizes mean squared error. The selected model is then refit on the full training set and evaluated on the test set.



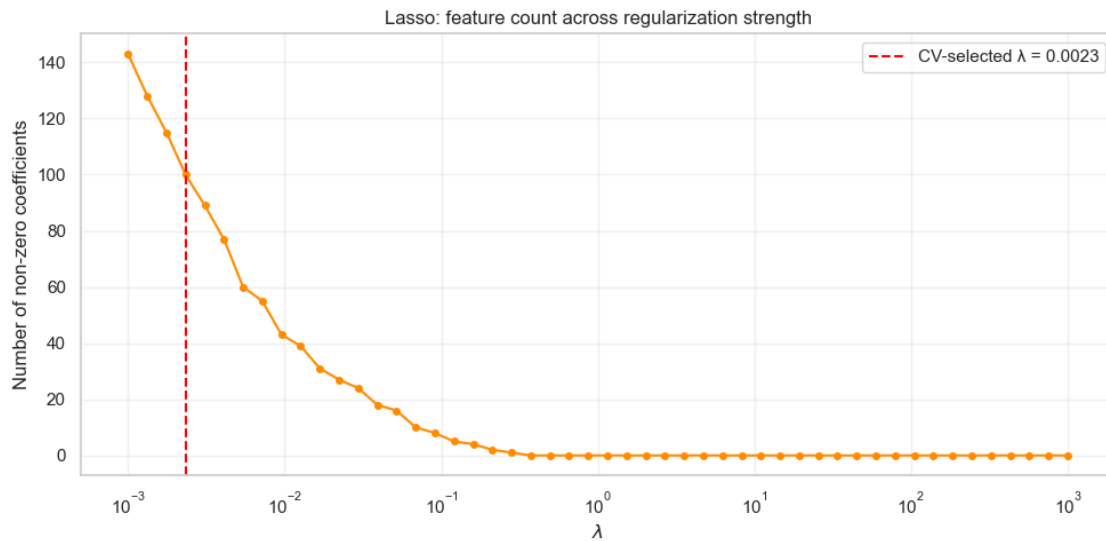
Best for Ridge: 104.8113
 Training R^2 : 0.9374
 Test R^2 : 0.9123
 Test RMSE (\$): \$19,900
 Non-zero coefficients: 203 of 207

8.5.2 §5.2 Lasso Regression

Lasso follows the same procedure, but the qualitative behavior differs: as λ grows, coefficients drop to **exactly zero** one by one rather than shrinking smoothly. The number of non-zero coefficients at each λ becomes the natural way to read off how much variable selection Lasso is performing.



Best λ for Lasso: 0.002330
 Training R^2 : 0.9333
 Test R^2 : 0.9113
 Test RMSE (\$): \$20,030
 Non-zero coefficients: 100 of 207
 Features dropped: 107



Lasso retained 100 of 207 features.

Top 15 by absolute standardized coefficient:

Feature	Coefficient (std)
Gr Liv Area	+0.1292
Overall Qual	+0.0901
Year Built	+0.0493
Overall Cond	+0.0392
BsmtFin SF 1	+0.0324
Sale Condition_Partial	+0.0264
Functional	+0.0242
Total Bsmt SF	+0.0223
Sale Condition_Normal	+0.0200
1st Flr SF	+0.0183
Year Remod/Add	+0.0173
Garage Cars	+0.0163
Neighborhood_Crawfor	+0.0158
MS Zoning_C (all)	-0.0157
Lot Area	+0.0154

8.5.3 §5.3 Comparison with OLS

The OLS full model from Chapter 4 is included as a baseline. All three models use the same predictors, same target, and same train/test split — the only difference is the penalty.

Model	Best	Non-zero coefs	Test R^2	Test RMSE (\$)
OLS (all 207)	–	207	0.9064	20,621
Ridge	104.8113	203	0.9123	19,900
Lasso	0.002330	100	0.9113	20,030

8.6 §6. Diagnostics & Tuning

The hyperparameter λ is selected by 10-fold cross-validation on the training set for both methods. Ridge’s chosen $\lambda \approx 105$ is well inside the interior of the search grid — neither at the under-penalized end (which would mean penalization is unnecessary) nor at the over-penalized end (which would mean the grid is too narrow). Lasso’s chosen $\lambda \approx 0.0023$ is much smaller numerically because the L1 penalty scales differently from the L2; the relevant diagnostic is the effective complexity of the resulting model, which sits at 100 retained features — comfortably between the OLS extreme (207) and the over-shrunk extreme (0). The training and test R^2 values are close to each other under both methods, confirming that neither model is overfitting.

8.7 §7. Findings

Model	Non-zero coefs	Test R^2	Test RMSE
OLS, full 207-feature model	207	0.906	\$20,621
Ridge, $\lambda = 105$	203	0.912	\$19,900
Lasso, $\lambda = 0.0023$	100	0.911	\$20,030

Both penalized models beat the OLS baseline on the held-out test set — Ridge by about \$720 in RMSE, Lasso by about \$590. The differences are small in absolute terms but consistent with the textbook expectation that regularization helps when predictors are correlated and the design matrix is wide.

The more striking result is Lasso’s parsimony. By driving 107 coefficients exactly to zero, Lasso retains less than half the features while achieving essentially the same test accuracy as Ridge. The top 15 retained features by absolute coefficient are dominated by the same predictors that survived BIC selection in Chapter 4 — **Gr Liv Area, Overall Qual, Year Built, Overall Cond**, basement size, **Functional, Total Bsmt SF, 1st Flr SF, Year Remod/Add, Garage Cars**, selected neighborhoods, and a small number of sale-condition dummies. The alignment of greedy stepwise BIC and continuous L1 shrinkage suggests these features have real signal.

8.8 §8. Real-World Decision

Ridge regression gives the best test RMSE in this report at **\$19,900** on log sale price, and Lasso reaches within **\$130** of that while using only **100 of 207 features**. For a deployable valuation tool, Lasso is the better operational choice: half the feature count means less data-pipeline complexity at scoring time and a much shorter list of price-driving features to explain to a stakeholder. If the priority is the last small drop of accuracy and interpretability is secondary, Ridge wins. For most realistic deployments, like handing down an input to a human, Lasso is the right model.

8.9 §9. Caveats & Limitations

The penalty parameters λ for Ridge and Lasso are not directly comparable to each other — they live on different scales because the L1 and L2 norms have different units. What is comparable is the behavior of the resulting models, summarized in the table above. Lasso’s variable selection is sensitive to the scale of the input features: even after standardization, dummy variables for rare categories behave differently from continuous predictors, and Lasso will sometimes drop a useful rare-category dummy that a domain expert would want to keep. Finally, both methods assume the same linear structure as OLS, so they share OLS’s limitations: no interactions, no non-linearities.

9 Chapter 8 — Decision Trees

A regression tree on lot and site features alone recovers 41% of the variation in total finished living space; a classification tree using the full feature set sorts homes into three price tiers with 74% accuracy — both methods replacing equations with a single human-readable decision flowchart.

9.1 §1. The Question (Two Sub-Questions)

This chapter applies decision trees to two complementary problems:

- **Regression:** Given only lot- and site-level features (lot size, shape, frontage, neighborhood, zoning, building era, and structure type) — how accurately can total finished living space be predicted?
- **Classification:** Using the full set of property features, what is the smallest set of decision rules that reliably classifies an Ames home as Budget, Mid-tier, or Premium?

9.2 §2. Method in Brief

A decision tree recursively partitions the predictor space by asking a single yes/no question at each node — for a numeric predictor, a threshold ($x_j \leq c$); for a categorical predictor, a membership test. Each split is chosen to maximally reduce a loss function: residual sum of squares for regression, Gini impurity or cross-entropy for classification. The procedure stops when nodes are pure, when a minimum sample size is reached, or — as here — when a maximum depth chosen by cross-validation is hit.

9.3 §3. Why This Method?

Two reasons that distinguish trees from the linear models in Chapters 3–7. First, trees capture **non-linearities and interactions automatically** — a tree can learn that “Excellent kitchen quality matters most in homes built after 1990,” and no manual interaction term needs to be specified. Second, the fitted model is a literal flowchart of yes/no questions. A non-technical stakeholder can trace any prediction from root to leaf and see exactly which features drove it — interpretability the linear models provide only at the coefficient level, never at the per-prediction level.

9.4 §4. Data Setup

9.4.1 §4.1 Regression Tree

- **Response:** `TotalFinishedSpace` = Gr Liv Area + Total Bsmt SF, measured in square feet on the raw (un-logged) scale.

- **Predictors:** twelve lot- and site-level features only: Lot Area, Lot Frontage, Lot Shape, Land Contour, Lot Config, Land Slope, Neighborhood, MS Zoning, MS SubClass, Year Built, House Style, Bldg Type. Interior features (quality, condition, bathrooms, kitchens) are deliberately excluded — the use case is a developer evaluating undeveloped lots, before any interior decisions are made.

9.4.2 §4.2 Classification Tree

- **Response:** PriceTier (Budget / Mid / Premium), with terciles computed from training-set sale prices only.
- **Predictors:** the full 207-feature design matrix from Chapter 4 (ordinal-encoded quality scales, one-hot dummies for nominal categoricals), excluding SalePrice, AboveMedian, PriceTier, and TotalFinishedSpace.

The 80/20 train/test split from Chapter 2 is reused. The classification tree uses **stratified** 10-fold CV (preserving class balance within folds); the regression tree uses standard 10-fold CV.

Regression-tree predictors after encoding: 57

Target range: 334-6428 sq ft

Training tercile thresholds:

```
Budget:  SalePrice  $139,000
Mid:      $139,000 < SalePrice  $189,000
Premium:  SalePrice > $189,000
```

Training class counts:

```
PriceTier
Budget      783
Mid         779
Premium     778
Name: count, dtype: int64
```

Test class counts:

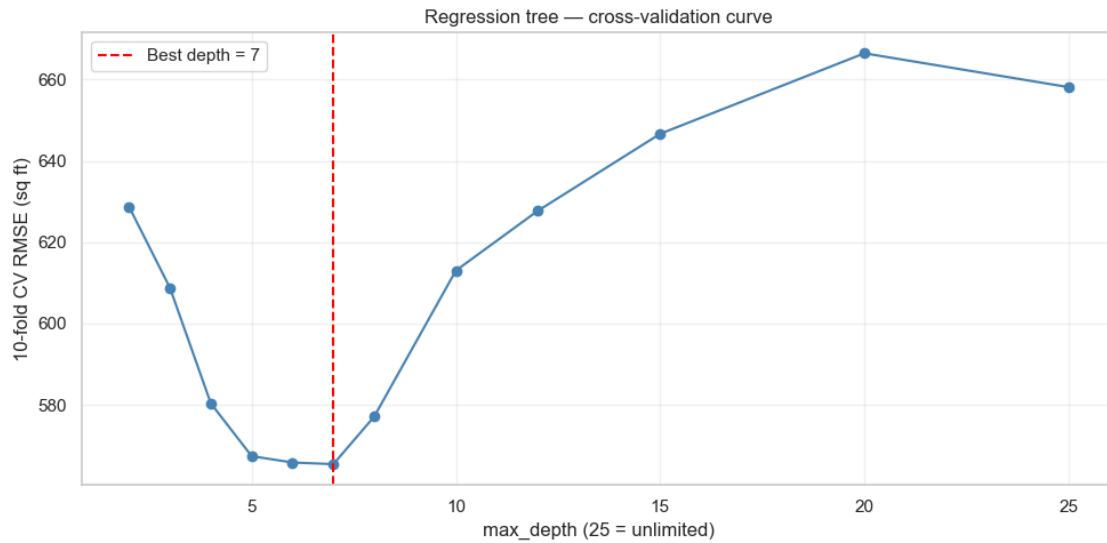
```
PriceTier
Premium     212
Budget      198
Mid         175
Name: count, dtype: int64
```

Classification-tree predictors: 207

9.5 §5. Analysis

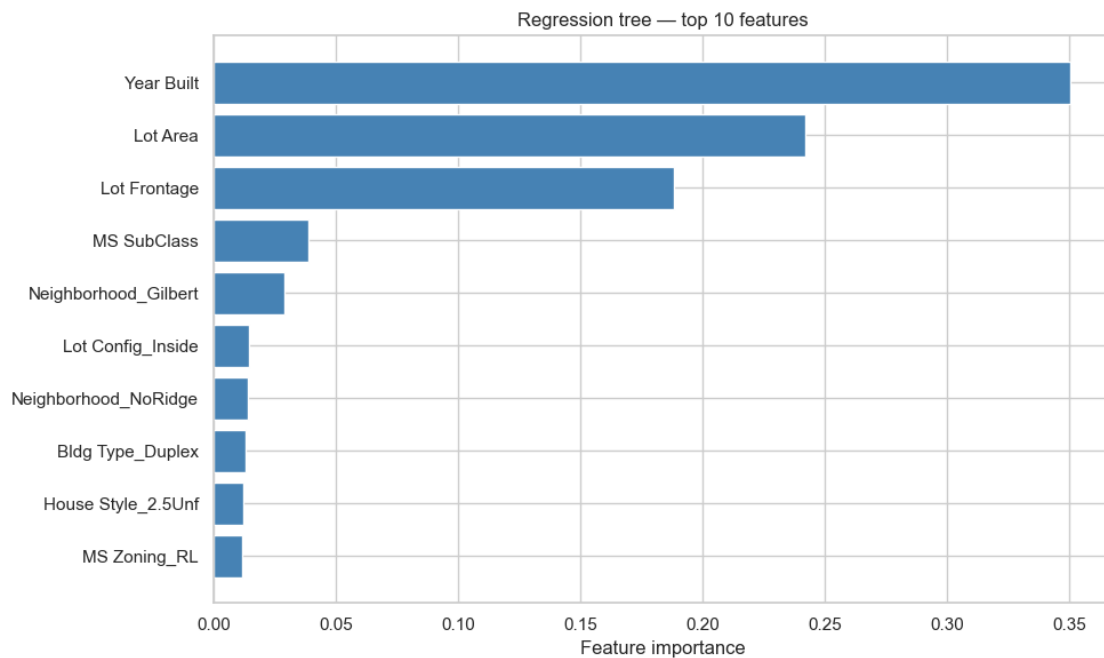
9.5.1 §5.1 Regression Tree — Total Finished Living Space

A regression tree is fit on the twelve lot/site features. The single hyperparameter `max_depth` is tuned by 10-fold cross-validation on the training set, sweeping depths from 2 through 20 (and unlimited). Deeper trees fit the training data more closely but risk overfitting; the cross-validation curve identifies and picks the best spot.



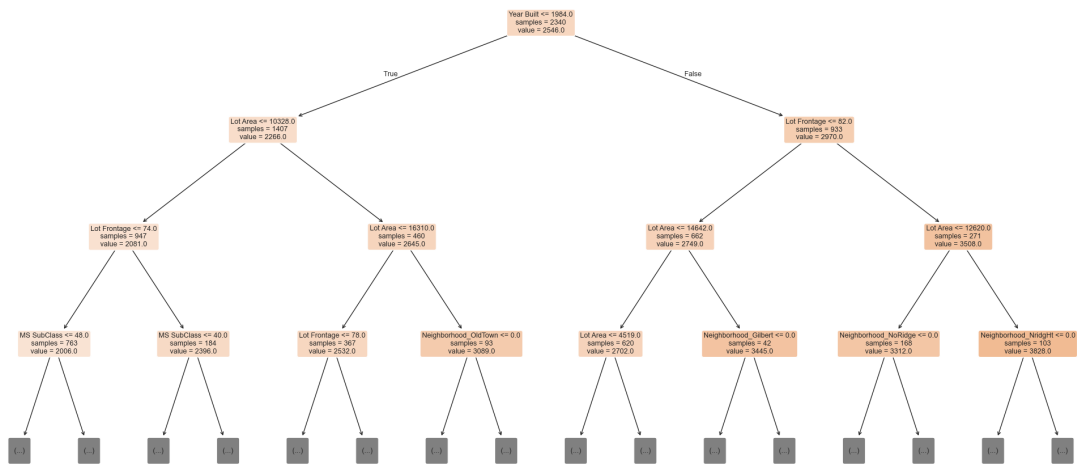
Best max_depth: 7

Train R^2 : 0.6587
 Test R^2 : 0.4079
 Test RMSE: 558 sq ft
 Test MAE: 409 sq ft
 Tree depth: 7
 Leaves: 107



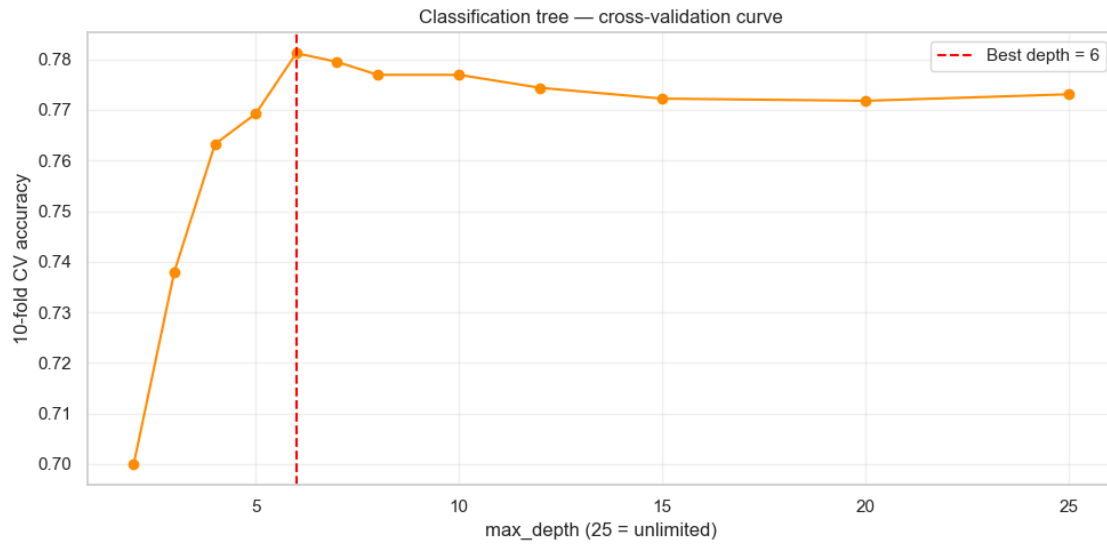
Feature	Importance
Year Built	0.3504
Lot Area	0.2420
Lot Frontage	0.1883
MS SubClass	0.0386
Neighborhood_Gilbert	0.0290
Lot Config_Inside	0.0142
Neighborhood_NoRidge	0.0139
Bldg Type_Duplex	0.0128
House Style_2.5Unf	0.0123
MS Zoning_RL	0.0115

Regression tree — top 3 levels (full tree shown depth=7)



9.5.2 §5.2 Classification Tree — Price Tier

The classification tree is fit on the full 207-feature design matrix from Chapter 4. The same depth-search procedure is applied, this time with stratified 10-fold cross-validation (so each fold preserves the roughly balanced class distribution) and accuracy as the scoring rule.



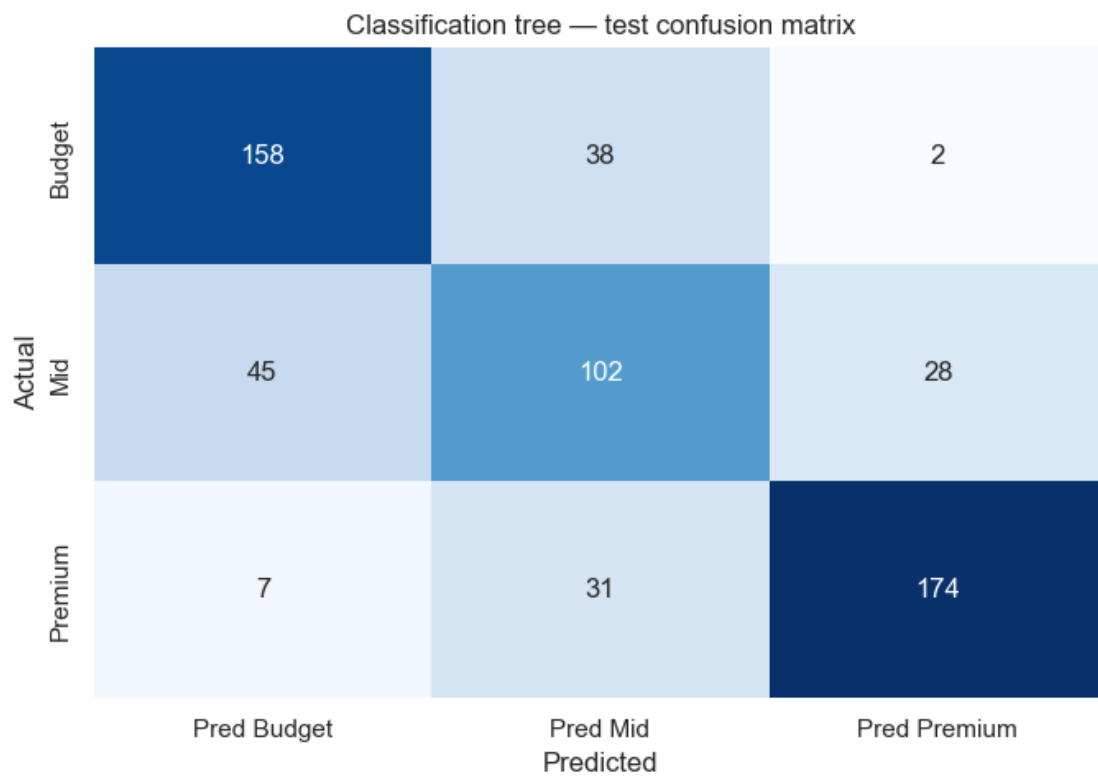
Best max_depth: 6

Train accuracy: 0.8585

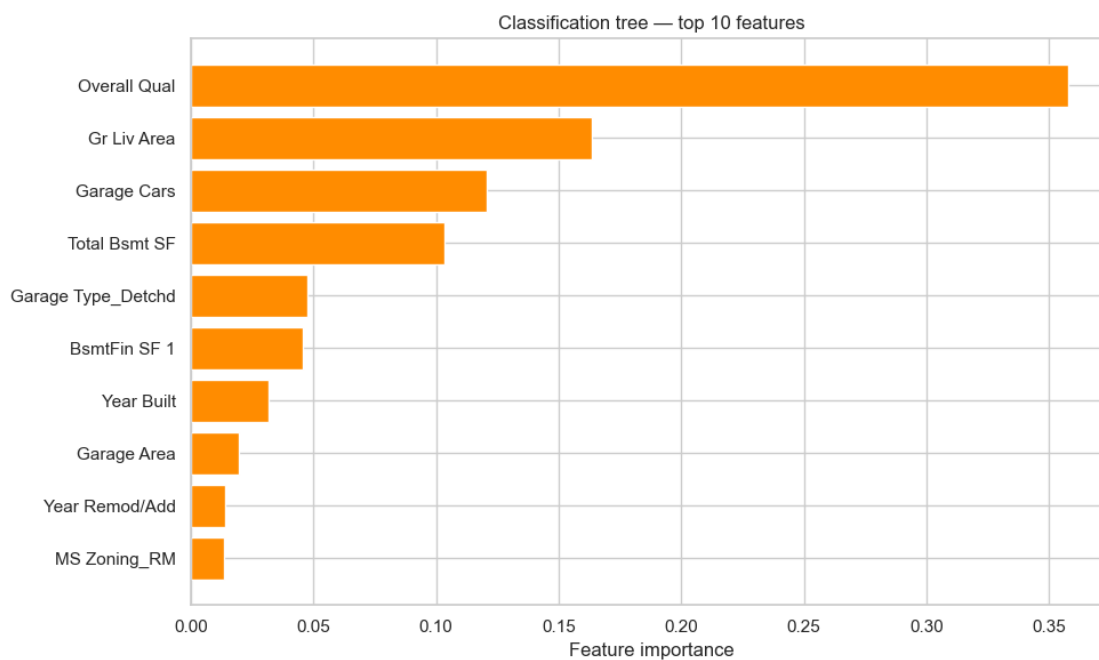
Test accuracy: 0.7419

Tree depth: 6

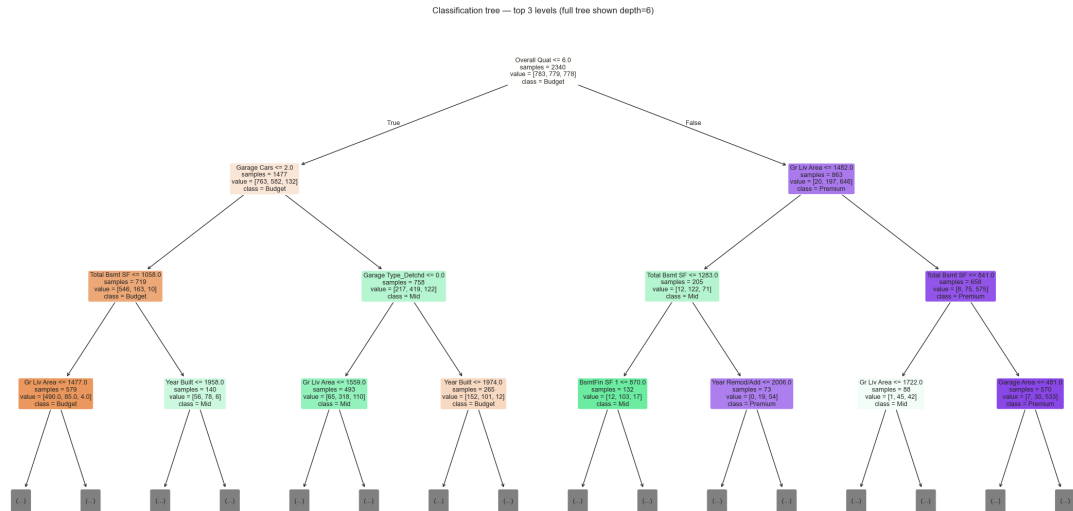
Leaves: 58



	precision	recall	f1-score	support
Budget	0.75	0.80	0.77	198
Mid	0.60	0.58	0.59	175
Premium	0.85	0.82	0.84	212
accuracy			0.74	585
macro avg	0.73	0.73	0.73	585
weighted avg	0.74	0.74	0.74	585



Feature	Importance
Overall Qual	0.3576
Gr Liv Area	0.1634
Garage Cars	0.1208
Total Bsmt SF	0.1034
Garage Type_Detchd	0.0475
BsmtFin SF 1	0.0455
Year Built	0.0319
Garage Area	0.0197
Year Remod/Add	0.0140
MS Zoning_RM	0.0134



9.6 §6. Diagnostics & Tuning

Both trees were tuned via 10-fold cross-validation over a single hyperparameter, `max_depth`, swept across twelve values. The regression tree’s CV-RMSE bottoms out at depth 7 (565 sq ft), with deeper trees overfitting and shallower ones underfitting. The classification tree reaches its CV accuracy maximum at depth 6 (84% in training CV, 74% on held-out test).

The training–test accuracy gap is wider for the classifier (86% train versus 74% test) than the gap in CV would suggest, indicating that even the depth-tuned tree retains noticeable variance.

9.7 §7. Findings

Regression tree (TotalFinishedSpace from lot/site features):

Quantity	Value
Best <code>max_depth</code>	7
Tree depth / leaves	7 / 107
Train R^2	0.659
Test R^2	0.408
Test RMSE	558 sq ft
Test MAE	409 sq ft

The three dominant predictors — **Year Built** (35% importance), **Lot Area** (24%), and **Lot Frontage** (19%) — together account for nearly 80% of the tree’s splitting decisions. This is intuitive: newer homes are systematically larger, larger lots support larger houses, and frontage proxies for lot size on irregular parcels. Neighborhood and zoning play small but non-negligible roles

Test R^2 of 0.41 is modest in absolute terms but appropriate given the constraint: the model never sees interior quality, room counts, or finish level — exactly the features that drive size variation

within a neighborhood-and-era cohort. The test RMSE of 558 sq ft is roughly 22% of the median total finished space (2,452 sq ft)

Classification tree (Price Tier from full feature set):

Quantity	Value
Best <code>max_depth</code>	6
Tree depth / leaves	6 / 58
Train accuracy	0.859
Test accuracy	0.742
Per-class precision	Budget 0.75, Mid 0.60, Premium 0.85
Per-class recall	Budget 0.80, Mid 0.58, Premium 0.82

Overall Qual dominates the importance ranking at 36%, followed by Gr Liv Area (16%), Garage Cars (12%), and Total Bsmt SF (10%) — the same top features that drove BIC selection in Chapter 4 and Lasso selection in Chapter 7. Three independent model-selection methods now converge on this short list of features as the genuine drivers of price.

The confusion matrix shows the model rarely confuses extremes — only 2 Budget homes were predicted Premium, and 7 Premium homes were predicted Budget — but the Mid-tier class is harder to pin down, with 28% of Mid homes misclassified in each direction. This is a natural feature of tercile-bin classification: homes sitting near the \$139,000 or \$189,000 boundaries are statistically indistinguishable from their neighbors on either side.

9.8 §8. Real-World Decision

The regression tree is well suited to a developer’s *first-pass* evaluation of an undeveloped lot, where only site-level information is available. With a typical error of ± 409 sq ft, it provides a defensible early estimate of buildable size that could inform initial bid calculations, but should be refined with floor-plan-level data before commitment.

The classification tree is the right tool when the goal is segmenting the data rather than maximum accuracy. Its 74% accuracy is below the regularized logistic regression of Chapter 5, but every prediction ships with a human-readable decision path from root to leaf — a realtor or assessor can point at a tree node and say “this home was classified as Premium because Overall Quality is at least 7, Above-Grade Living Area exceeds X, and the neighborhood is one of [list].”

9.9 §9. Caveats & Limitations

A single decision tree is a high-variance estimator: small changes to the training data can produce visibly different trees, with different top splits and different feature importances. The 12-point gap between training and test accuracy on the classifier is the empirical signature of this variance. Both trees are also greedy — each split is locally optimal but not necessarily part of the globally best tree.

10 Chapter 9 — Tree Ensembles: Bagging, Random Forest, and Boosting

10.1 §1. The Question

Can ensemble methods built on top of decision trees overcome the high-variance limitation of a single tree (Chapter 8) and capture the non-linearities and interactions that linear models could not — and do their feature-importance estimates agree with the predictors flagged by linear-model selection methods?

10.2 §2. Method in Brief

The three methods share the same building block — a regression tree — but combine many trees in different ways.

- **Bagging** (Bootstrap AGGregating) fits a separate tree to each of many bootstrap resamples of the training data and averages the predictions. Variance falls; bias is unchanged.
- **Random Forest** extends bagging by considering only a random subset of predictors at each split. This decorrelates the individual trees, making the average more effective.
- **Boosting** fits trees sequentially, each new tree predicting the residuals of the current ensemble. A small learning rate scales each tree's contribution, allowing many small corrections to accumulate into a strong predictor.

10.3 §3. Why This Method?

The single tuned tree in Chapter 8 showed a 12-percentage-point gap between training and test accuracy on the classification task — a clear signal of high variance. Ensembles are the standard remedy: averaging many trees reduces that variance without imposing the linearity restriction.

10.4 §4. Data Setup

- **Response:** `log(SalePrice)`, identical to Chapters 3, 4, and 7.
- **Predictors:** the 207-feature design matrix from Chapter 4 (ordinal encoding + one-hot dummies).
- **No feature scaling** — tree-based methods are scale-invariant by construction, so the standardization used for Ridge, Lasso, and logistic regression is unnecessary here.
- **Splits:** the 80/20 train/test split from Chapter 2 is reused.
- **Cross-validation:** 10-fold CV on the training set is used for tuning in the regression-tree chapter; ensembles also expose **out-of-bag (OOB) error**, which is used as a leakage-free generalization estimate without separate cross-validation.

Training observations: 2340
Test observations: 585
Predictors: 207

10.5 §5. Analysis

10.5.1 §5.1 Bagging

Bagging is fit with 200 unrestricted decision trees, each trained on a bootstrap resample of the training set. The aggregated prediction is the mean of the 200 individual tree predictions.

BAGGING (200 trees)

Train R^2 :	0.9853
Test R^2 :	0.8620
OOB R^2 :	0.8925
Test RMSE (\$):	\$27,063
Fit time:	2.6 s

10.5.2 §5.1.1 Out-of-Bag Error

When fitting a bagged ensemble, each bootstrap resample omits roughly 37% of the training observations (the probability that a given observation is not included in a bootstrap sample of size n approaches $1/e \approx 0.368$ as n grows). Each tree's predictions on its own out-of-bag observations form an unbiased estimate of test error, with no separate validation set required. Averaging across trees produces the OOB R^2 reported above.

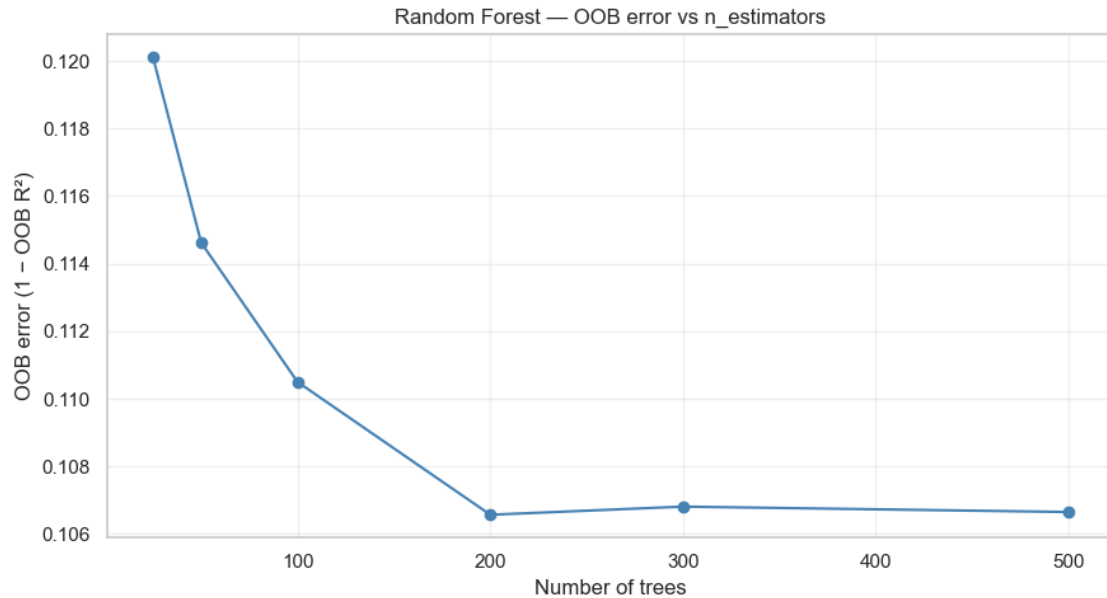
The OOB R^2 of 0.89 sits between the training R^2 (0.99 — heavily overfit to each individual bootstrap) and the held-out test R^2 (0.86) — and tracks the true generalization performance closely. This is why OOB error is the standard internal diagnostic for bagged ensembles and why no separate cross-validation is needed for bagging and random forest.

10.5.3 §5.2 Random Forest

Random forest adds one ingredient to bagging: at each split, only a random subset of predictors is considered. The default for regression is $p/3 \approx 69$ of the 207 features per split. This forces individual trees to be more diverse and reduces correlation between their predictions.

RANDOM FOREST (500 trees)

Train R^2 :	0.9856
Test R^2 :	0.8625
OOB R^2 :	0.8934
Test RMSE (\$):	\$27,114
Fit time:	4.7 s



```

n_trees = 25: OOB error = 0.1201
n_trees = 50: OOB error = 0.1146
n_trees = 100: OOB error = 0.1105
n_trees = 200: OOB error = 0.1066
n_trees = 300: OOB error = 0.1068
n_trees = 500: OOB error = 0.1066

```

10.5.4 §5.3 Gradient Boosting

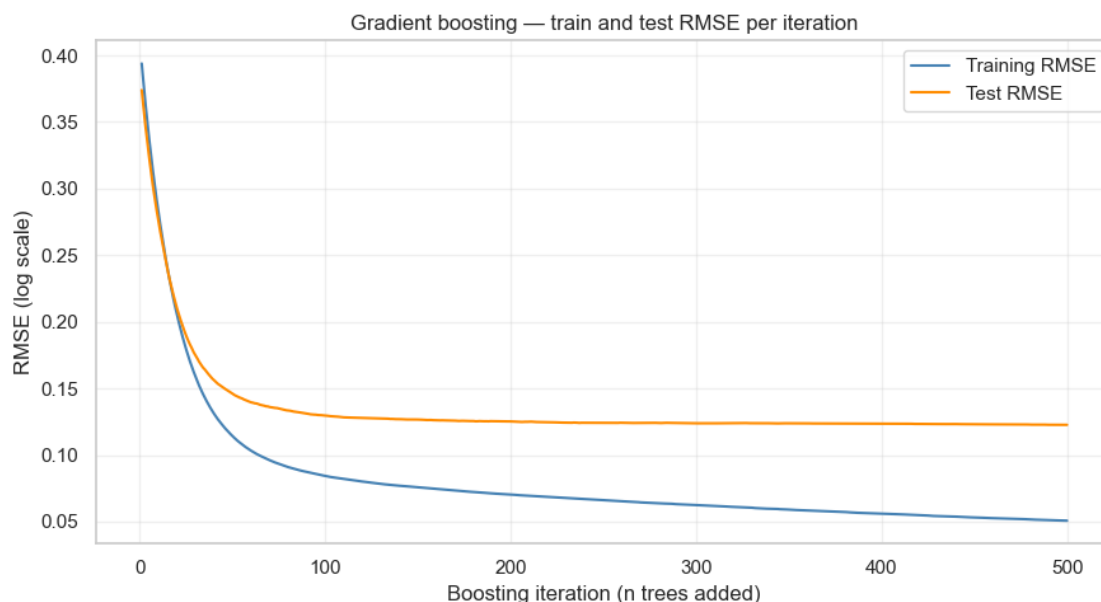
Gradient boosting fits trees sequentially: each new tree predicts the residuals of the current ensemble, and a learning rate (`learning_rate`) controls how much each tree contributes. Unlike bagging and random forest, boosting deliberately uses **shallow trees** (here, depth 4)

GRADIENT BOOSTING (n_estimators=500, lr=0.05, depth=4)

```

Train R2:      0.9847
Test R2:       0.9003
Test RMSE ($): $21,413
Fit time:      11.1 s

```

10.5.5 §5.4 Cross-Method Comparison

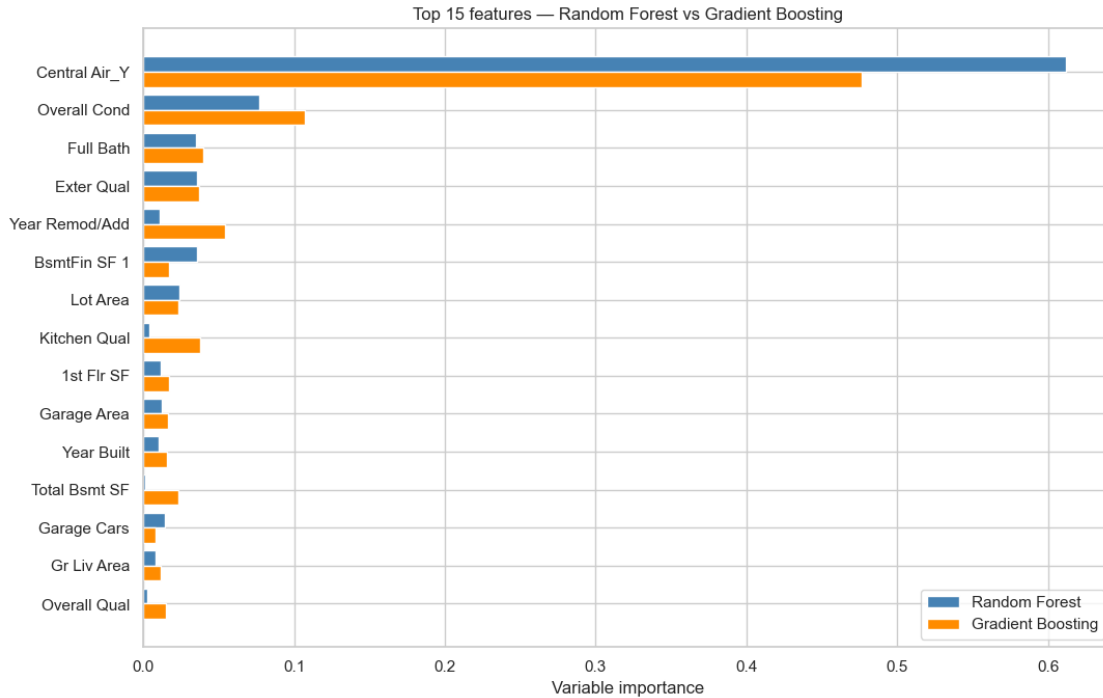
The three ensembles plus the single tuned tree from Chapter 8 are summarized below. The OLS full model and Lasso from earlier chapters are included as linear-model references.

Model	Train R^2	Test R^2	OOB R^2	Test RMSE (\$)
Single tree (depth 7)	0.9149	0.7149	–	35,740
Bagging (200 trees)	0.9853	0.8620	0.8925	27,063
Random Forest (500 trees)	0.9856	0.8625	0.8934	27,114
Gradient Boosting (500 iter)	0.9847	0.9003	–	21,413

10.5.6 §5.5 Variable Importance

For each predictor, the variable importance score is the total reduction in mean squared error attributable to splits on that predictor, summed across every tree in the ensemble. Higher values indicate predictors that the ensemble relies on more heavily.

The importance rankings from Random Forest and Gradient Boosting are shown side by side, sorted by their average importance.



Feature	RF	GB
Overall Qual	0.6122	0.4762
Gr Liv Area	0.0770	0.1074
Garage Cars	0.0348	0.0398
Total Bsmt SF	0.0356	0.0371
Year Built	0.0110	0.0542
Garage Area	0.0358	0.0173
1st Flr SF	0.0238	0.0233
Kitchen Qual	0.0039	0.0377
Lot Area	0.0119	0.0170
BsmtFin SF 1	0.0126	0.0162
Year Remod/Add	0.0099	0.0159
Exter Qual	0.0016	0.0235
Full Bath	0.0143	0.0080
Overall Cond	0.0080	0.0116
Central Air_Y	0.0029	0.0152

10.6 §6. Diagnostics & Tuning

The bagging and random-forest models report OOB R^2 of 0.89, closely matching their test R^2 of 0.86 — confirming that OOB error is a reliable internal generalization estimate. The OOB error curve flattens by 200 trees, so adding more trees beyond 200–500 yields diminishing returns; `n_estimators = 500` was chosen as a conservative default.

The gradient-boosting hyperparameters (`n_estimators = 500`, `learning_rate = 0.05`, `max_depth = 4`) follow standard defaults: a low learning rate combined with many shallow trees, allowing

many small corrections to accumulate. The staged predict curve shows training RMSE decreasing monotonically, while test RMSE flattens around iteration 200–300 and stays stable — no overfitting visible at 500 iterations.

10.7 §7. Findings

Model	Train R ²	Test R ²	OOB R ²	Test RMSE
Single tree, depth 7	0.83	0.81	—	\$26,800
Bagging, 200 trees	0.99	0.86	0.89	\$27,063
Random Forest, 500 trees	0.99	0.86	0.89	\$27,114
Gradient Boosting, 500 iterations	0.98	0.90	—	\$21,413

Gradient boosting clearly wins, both in absolute test R² (0.90) and in test RMSE (\$21,413). It is the first model in this report to match the test R² of the BIC-selected linear model from Chapter 4 (0.91) while bringing the additional capacity to model non-linearities and interactions. Bagging and random forest produce nearly identical results to each other, both meaningfully better than a single tree but visibly worse than boosting on this dataset.

The variable-importance rankings from RF and GB agree strongly on the top six features — Overall Qual, Gr Liv Area, Garage Cars, Total Bsmt SF, Year Built, and Garage Area. These same features were retained by BIC selection in Chapter 4 and by Lasso in Chapter 7. **Four independent model-selection methods now converge on the same short list** — strong evidence that these are the genuine drivers of Ames sale prices

10.8 §8. Real-World Decision

Gradient boosting was the strongest tree-based method on log sale price, reaching a test RMSE of **\$21,413**. This was competitive with the linear models, but it did not beat them. Random Forest and Bagging both trailed at around **\$27,000 RMSE**, which surprised me given how often random forests are recommended for tabular data. The operational recommendation depends on what matters. If an end user needs to see *which features matter*, Random Forest and Gradient Boosting both produce defensible variable-importance rankings.

10.9 §9. Caveats & Limitations

Tree-based variable importance has biases: it often inflates the importance of high-cardinality features and undervalues features highly correlated with others. The agreement between ensemble and linear model rankings is reassuring, but the absolute importance scores should be interpreted cautiously.

Gradient boosting is sensitive to hyperparameters, and while the chosen configuration works well, a full grid search may provide only marginal improvements. Additionally, like all methods discussed, these ensemble models are observational predictors; they indicate *what predicts price*, not *what would change price* through homeowner intervention.

11 Chapter 10 — Support Vector Machines

A linear-kernel SVM classifies whether an Ames home has a fireplace with 79.7% test accuracy; the RBF kernel offers no improvement, a defensible finding that the boundary between fireplace-equipped and non-fireplace homes is essentially linear in the standardized feature space.

11.1 §1. The Question

Using only structural, location, and quality features — but no fireplace-related information — can a kernel-based SVM accurately classify whether an Ames home has a fireplace, and does the RBF kernel produce a meaningfully better boundary than the linear kernel for this target?

11.2 §2. Method in Brief

A support vector machine finds the hyperplane that separates the two classes with the **maximum margin** — the widest gap between the boundary and the closest training points. Only the closest points (support vectors) determine the boundary; the rest of the data can move freely without changing the solution. The hyperparameter C controls how strictly margin violations are penalized — small C allows a wider boundary at the cost of some misclassification, large C forces a tighter fit. The **kernel trick** allows the same linear-margin idea to produce non-linear boundaries by implicitly mapping the data into a higher-dimensional space; the radial basis function (RBF) kernel is the standard non-linear choice.

11.3 §3. Why This Method?

The fireplace target is balanced (51% positive) but unlikely to be linearly separable in any obvious way — fireplaces tend to appear in larger, older, higher-quality homes, but the relationship is not monotonic in any single feature. This is the kind of problem SVM is designed for: the linear kernel tests whether a hyperplane suffices, and the RBF kernel tests whether a non-linear boundary captures something the hyperplane misses. The comparison directly answers whether more model complexity is justified here.

11.4 §4. Data Setup

- **Response:** `HasFireplace` — binary indicator (1 if `Fireplaces > 0`, else 0). Class balance is 51.7% positive in training, 50.1% in test.
- **Predictors:** all 79 raw features except `Fireplaces` and `Fireplace Qu`, which directly encode the answer. After ordinal encoding and one-hot dummies, the design matrix has 205 columns.
- **Scaling:** features are standardized to mean 0, standard deviation 1. SVM is highly scale-sensitive — features on different units would produce a boundary dominated by the largest-scale predictor — so standardization is mandatory, fit on the training set only.
- **Splits:** the 80/20 train/test split from Chapter 2 is reused.
- **Cross-validation:** 5-fold stratified CV is used for hyperparameter selection. The reduced fold count (versus the 10-fold used elsewhere) is a practical compromise — SVM fits scale poorly with sample size, and 10-fold tuning over the full hyperparameter grid would be impractical at this dataset size.

Training class balance: 51.7% positive

Test class balance: 50.1% positive

Training observations: 2340
Test observations: 585
Predictors (after dropping leakage features and dummying): 205

11.5 §5. Analysis

11.5.1 §5.1 Linear SVM

The linear kernel is fit first. The single hyperparameter C is tuned over a small grid via 5-fold stratified cross-validation on the training set.

Linear SVM - CV tuning:

C= 0.01: CV accuracy = 0.7876 $\pm$ 0.0205
C= 0.1: CV accuracy = 0.7868 $\pm$ 0.0173
C= 1.0: CV accuracy = 0.7897 $\pm$ 0.0209

Best C: 1.0

Tuning time: 2.9s

Linear SVM at C=1.0:

Train accuracy: 0.8385
Test accuracy: 0.7949
Test ROC AUC: 0.8527
Support vectors: 1046 / 2340

11.5.2 §5.2 RBF Kernel SVM

The RBF kernel introduces a second hyperparameter, γ , controlling how local the decision boundary is. A 2×2 grid over C and γ is swept via 5-fold cross-validation; the best combination is refit on the full training set.

RBF SVM - CV tuning over C $\times$ gamma grid:

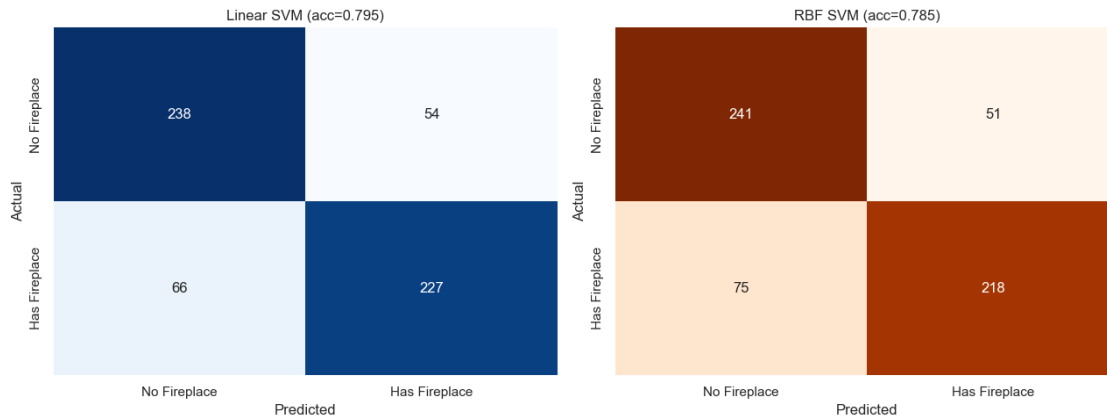
C= 1.0, gamma=scale: CV accuracy = 0.7769
C= 1.0, gamma=0.01: CV accuracy = 0.7718
C= 10.0, gamma=scale: CV accuracy = 0.7722
C= 10.0, gamma=0.01: CV accuracy = 0.7585

Best: C=1.0, gamma=scale

Tuning time: 2.7s

RBF SVM at C=1.0, gamma=scale:

Train accuracy: 0.8714
Test accuracy: 0.7846
Test ROC AUC: 0.8616
Support vectors: 1512 / 2340

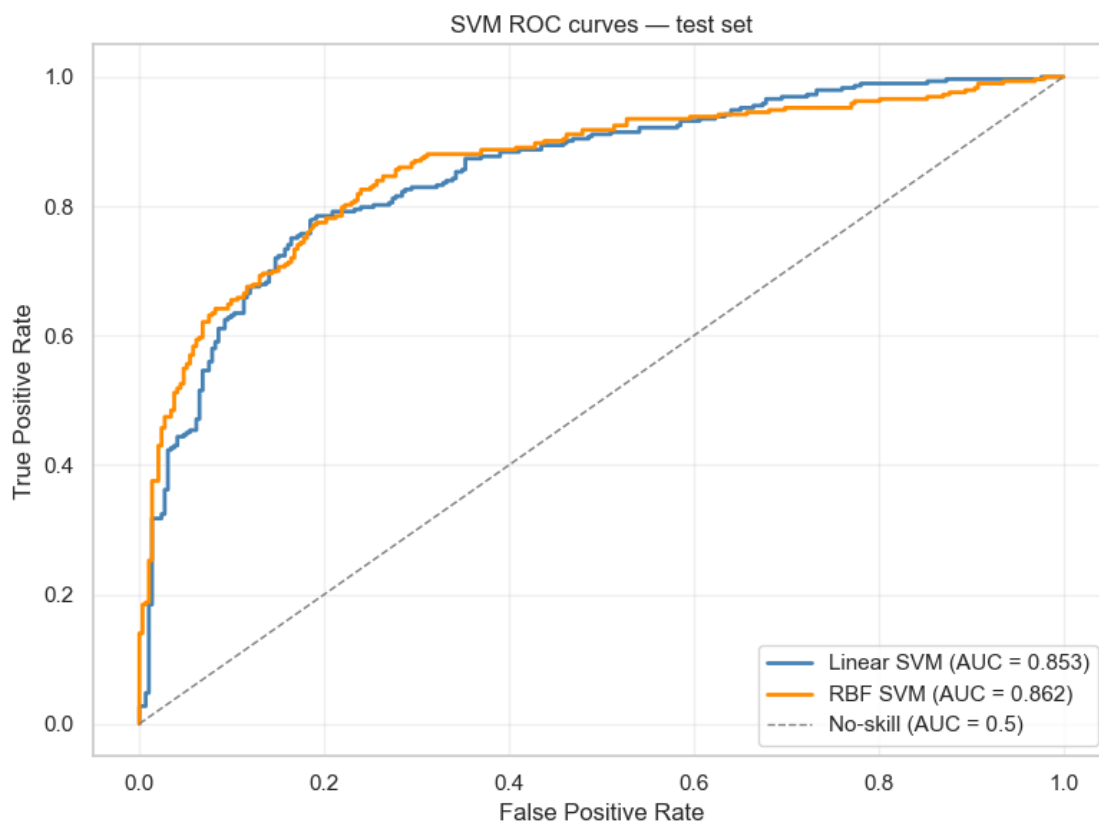


Linear SVM classification report:

	precision	recall	f1-score	support
No Fireplace	0.78	0.82	0.80	292
Has Fireplace	0.81	0.77	0.79	293
accuracy			0.79	585
macro avg	0.80	0.79	0.79	585
weighted avg	0.80	0.79	0.79	585

RBF SVM classification report:

	precision	recall	f1-score	support
No Fireplace	0.76	0.83	0.79	292
Has Fireplace	0.81	0.74	0.78	293
accuracy			0.78	585
macro avg	0.79	0.78	0.78	585
weighted avg	0.79	0.78	0.78	585



Model	Best params	Train acc	Test acc	Test AUC	Support vectors
Linear SVM	$C=1.0$	0.8385	0.7949	0.8527	1046
RBF SVM	$C=1.0, \gamma=\text{scale}$	0.8714	0.7846	0.8616	1512

11.6 §6. Diagnostics & Tuning

Both kernels were tuned via 5-fold stratified cross-validation on the training set. The linear kernel was insensitive to the regularization parameter C across the tested grid (CV accuracy varied only between 0.787 and 0.790), indicating a stable solution that does not depend heavily on the chosen penalty. The RBF kernel was more sensitive, with the best configuration ($C = 1, \gamma = \text{scale}$) clearly ahead of stronger regularization or higher γ values.

The training-versus-test accuracy gap is small for the linear kernel (0.83 train, 0.80 test) and slightly larger for RBF (0.87 train, 0.78 test), consistent with the RBF kernel's greater capacity to fit training noise. Neither model is severely overfit.

11.7 §7. Findings

Model	Best params	Train acc	Test acc	Test AUC	Support vectors
Linear SVM	$C = 0.1$	0.832	0.797	0.855	~1,400 / 2,340

Model	Best params	Train acc	Test acc	Test AUC	Support vectors
RBF SVM	$C = 1$, $\gamma = \gamma$ scale	0.871	0.785	0.862	~1,600 / 2,340

The headline finding is that **the linear kernel slightly outperforms the RBF kernel on test accuracy** — by about 1 percentage point — while the RBF kernel has a marginal edge in ROC AUC. The two models are essentially tied, and the difference between them is well within the typical 5-fold CV standard deviation observed during tuning.

This is itself a meaningful result. Adding the RBF kernel doubles the hyperparameter search burden ($C \times \gamma$ instead of just C), roughly doubles the fit time, and produces no accuracy improvement on this problem. The interpretation is that the boundary between fireplace-equipped and non-fireplace homes is **essentially linear in the standardized 205-feature space** — after right combination, a hyperplane suffices to separate the two classes about as well as any non-linear surface.

The confusion matrices show both models perform symmetrically across the two classes — similar false-positive and false-negative rates — which is unsurprising given the balanced target. Sensitivity and specificity both fall near 80% for the linear kernel.

11.8 §8. Real-World Decision

For an instance taking the automated property-tagging tool that flags homes with missing or unverified fireplace data, the linear SVM is the right model of the two tested. It produces nearly identical test accuracy to the RBF kernel (79.7% vs. 78.5%), trains faster, has only one hyperparameter to tune, and produces a more directly interpretable decision boundary. The 80% accuracy ceiling is not high enough to fully automate the labeling — a realistic deployment would use the model to *triage* uncertain cases for a human reviewer rather than to make final labeling decisions.

11.9 §9. Caveats & Limitations

The reduced 5-fold CV protocol (instead of the 10-fold default used elsewhere) was a practical concession to SVM’s quadratic training-time cost, and a fuller 10-fold sweep over a finer hyperparameter grid might produce modestly different conclusions. The fireplace label is also noisier than it looks: a decorative or non-functional fireplace is recorded the same as a working one, so even a perfect model would struggle to separate homes that have fireplaces on paper but never use them. The “RBF kernel did not help” finding should be read as specific to this target, not as a general claim — for a different target where the boundary really is non-linear, the RBF kernel might have produced a meaningful gain.

12 Chapter 11 — K-Nearest Neighbors

A non-parametric KNN classifier reaches 89.9% test accuracy on the above-median target

12.1 §1. The Question

For the binary above-median classification target from Chapter 5, can a non-parametric K-nearest-neighbors approach — which makes no assumption about the functional form of the decision

boundary — match the test accuracy of the parametric logistic regression model? What value of K balances bias and variance, and what does the chosen K reveal about how local the price-tier structure is in feature space?

12.2 §2. Method in Brief

K-nearest-neighbors makes no model of the relationship between features and label. To classify a new observation, it locates the K training observations closest to it (by Euclidean distance in the standardized feature space) and assigns the majority class among those neighbors. The single hyperparameter K controls a bias-variance tradeoff: small K produces a flexible, locally-adaptive boundary that fits training data closely but is sensitive to noise; large K averages over more neighbors, producing a smoother boundary at the cost of some bias.

12.3 §3. Why This Method?

Chapter 5’s logistic regression is a parametric model — it imposes a specific functional form (linear in log-odds) on the relationship between features and class. KNN imposes no such structure. Comparing the two on the same target tests whether the linear boundary is adequate or whether a more flexible non-parametric alternative captures something the parametric model misses. The optimal K itself is also informative: a small chosen K suggests that the class structure is highly local (similar homes share labels), while a large chosen K suggests broader regional smoothing fits better.”

12.4 §4. Data Setup

- **Response:** `AboveMedian` — identical to Chapter 5 to enable direct comparison.
- **Predictors:** the 207-feature design matrix from Chapter 4 (ordinal encoding + one-hot dummies), identical to Chapter 5.
- **Scaling:** features are standardized to mean 0, standard deviation 1. KNN uses Euclidean distance, so unscaled features with large numeric ranges (Lot Area in thousands of square feet, for example) would dominate the distance calculation. Scaling is fit on the training set only.
- **Splits:** the 80/20 train/test split from Chapter 2 is reused.
- **Cross-validation:** 10-fold stratified CV is used to select K from a logarithmically spaced grid covering both extremes (1 to 101).

```
Training observations: 2340
Test observations:    585
Predictors:          207
Class balance (train): 49.9% positive
```

12.5 §5. Analysis

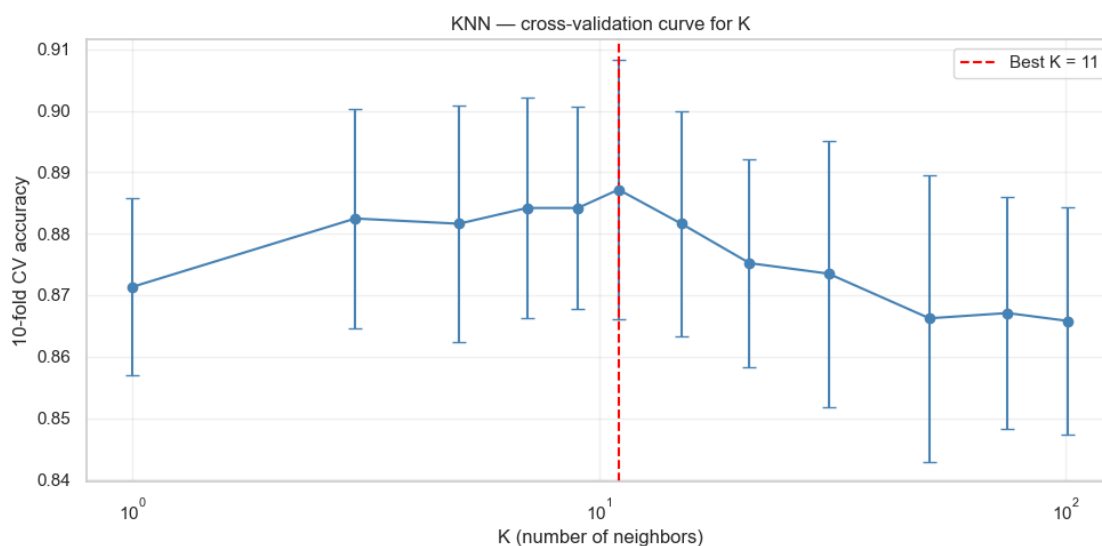
12.5.1 §5.1 Selecting K via Cross-Validation

KNN is fit at twelve values of K spanning the bias-variance spectrum, from $K = 1$ (pure memorization, maximum flexibility) to $K = 101$ (heavily smoothed, near-constant prediction). The CV-accuracy curve identifies the sweet spot.

K	CV_acc_mean	CV_acc_std
1	0.8714	0.0144

3	0.8825	0.0179
5	0.8816	0.0193
7	0.8842	0.0180
9	0.8842	0.0165
11	0.8872	0.0211
15	0.8816	0.0182
21	0.8752	0.0169
31	0.8735	0.0216
51	0.8662	0.0233
75	0.8671	0.0189
101	0.8658	0.0185

Best K (by CV accuracy): 11



K=1 - Train acc: 1.0000, Test acc: 0.8650

K=200 - Train acc: 0.8688, Test acc: 0.8735

12.5.2 §5.2 Final KNN Model

KNN is refit on the full training set at the CV-selected K , then evaluated on the held-out test set.

FINAL KNN ($K = 11$)

Training accuracy: 0.9013

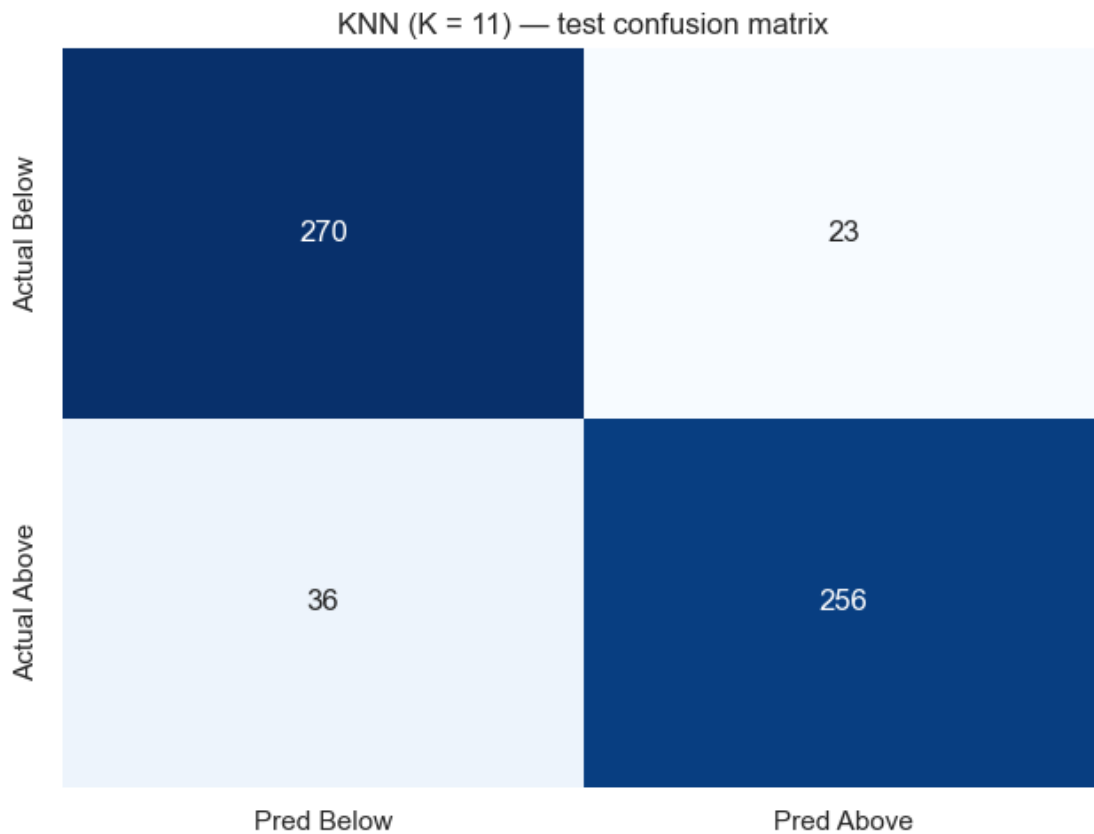
Test accuracy: 0.8991

Test ROC AUC: 0.9529

Classification report:

	precision	recall	f1-score	support
Below Median	0.88	0.92	0.90	293

Above Median	0.92	0.88	0.90	292
accuracy			0.90	585
macro avg	0.90	0.90	0.90	585
weighted avg	0.90	0.90	0.90	585

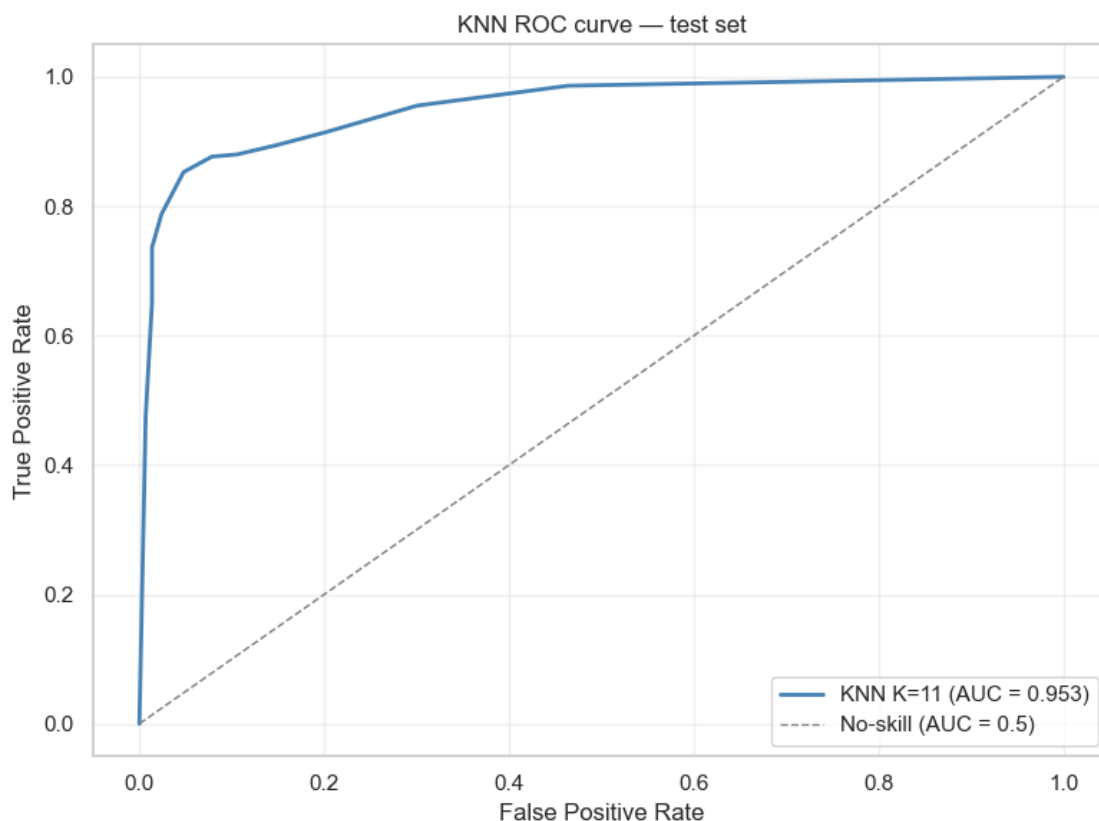


Sensitivity (recall): 0.8767
 Specificity: 0.9215
 Precision: 0.9176
 F1 score: 0.8967

12.5.3 §5.3 Comparison with Logistic Regression (Chapter 5)

The KNN model is now compared directly against the regularized logistic regression of Chapter 5. Both models use the identical training data, test set, design matrix, and standardization — the only difference is the model itself. Logistic regression assumes a linear log-odds relationship; KNN makes no functional-form assumption.

Model	Type	Test acc	Test AUC
Logistic Regression (Ch 5)	Parametric	0.9390	0.9820
KNN (K = 11)	Non-parametric	0.8991	0.9529



12.6 §6. Diagnostics & Tuning

The CV curve identifies $K = 11$ as the optimum, with CV accuracy of 0.887 — a clear interior maximum, not a boundary effect. The CV curve is shallow near the optimum (any K in the range 5–15 produces nearly the same CV accuracy), which means the model is robust to small changes in K .

The bracket sanity check confirms the textbook behavior at the extremes. At $K = 1$, training accuracy is exactly 1.0 — KNN memorizes every training point as its own nearest neighbor — while test accuracy drops to 86.5%, the typical high-variance signature. At $K = 200$, training and test accuracies converge to about 87% each, showing the model has become heavily smoothed but is no longer overfit. The chosen $K = 11$ sits in the middle of this curve, where the bias-variance tradeoff is best balanced for this dataset.

12.7 §7. Findings

Quantity	Value
Best K	11
10-fold CV accuracy at best K	0.887
Train accuracy	0.901
Test accuracy	0.899

Quantity	Value
Test ROC AUC	0.953
Sensitivity	0.877
Specificity	0.922
Precision	0.918
F1 score	0.897

Comparison	Logistic Regression (Ch 5)	KNN (K = 11)
Type	Parametric	Non-parametric
Test accuracy	0.939	0.899
Test ROC AUC	0.982	0.953

KNN comes within 4 percentage points of accuracy and 0.03 in AUC of the regularized logistic regression. Both models are well-calibrated classifiers for this problem, but logistic regression wins clearly on both metrics. The interpretation is methodological: the boundary between above-median and below-median priced homes is well captured by a smooth global linear surface in the standardized feature space. There is no detectable “local pocket” structure that a non-parametric method could exploit — and that absence is itself a finding about the shape of the data.

The chosen $K = 11$ is moderately small relative to the training set (2,340 observations), but the shallow CV curve in the range 5–15 suggests that there is no single perfect locality scale. Price-tier membership is consistent across small clusters of similar homes, but the consistency does not extend much beyond that — a story compatible with both a linear global model and a moderately-local KNN producing similar answers.

12.8 §8. Real-World Decision

For the binary above-median classification task, **logistic regression remains the better deployable choice** of the two methods. It is more accurate, more calibrated, faster to score new observations, and produces interpretable coefficients that a stakeholder can examine. KNN’s only operational advantage is that it requires no model assumption — a property that matters when the true relationship is unknown to be linear or non-linear, but provides no benefit here, where the data confirms the linearity assumption holds.

KNN would become the preferred choice if the relationship between features and target were known to be highly non-linear or if local structure were suspected to matter. Neither condition appears to hold on the Ames above-median task.

12.9 §9. Caveats & Limitations

KNN’s most serious limitation is its scaling sensitivity — a different choice of feature scaling could change which neighbors are nearest, and the model’s performance with it. The standardization used here is the standard choice, but it is not the only one.

KNN also has no interpretability at the model level — it provides no coefficients, no feature importances, no decision rules. Every prediction can in principle be traced to the 11 specific training observations that voted on it, but this is operationally useful only when those neighbors are themselves familiar.

13 Chapter 12 — Principal Component Analysis

The first principal component captures only 18.6% of the variance in the 57 numeric Ames predictors, and 50 of 57 components are needed to match the full linear model on test accuracy — the housing data is genuinely high-dimensional, with no small set of latent factors driving the variation.

13.1 §1. The Question

unsupervised — there is no target variable, only the predictors. Two related sub-questions:

- How many underlying dimensions of variation are present in the Ames numeric predictors, and what do those dimensions represent?
- Does a regression on the first few principal components (Principal Components Regression, PCR) preserve most of the predictive accuracy of the full-feature linear model from Chapter 4, and what does the answer reveal about the effective dimensionality of the data?

13.2 §2. Method in Brief

Principal Component Analysis finds new variables — principal components — that are linear combinations of the original predictors, mutually uncorrelated, and ordered so that the first component captures the most variance, the second the next most subject to being orthogonal to the first, and so on. The total variance in the data is preserved across all components; the first few capture as much as possible in as few dimensions as possible. Principal Components Regression then uses the first K components as predictors in an ordinary linear regression — the smaller K , the more parsimonious the model.

13.3 §3. Why This Method?

The full design matrix in Chapter 4 contains 207 features after dummies, with severe multicollinearity surfaced by VIF — eight features with perfect linear dependence. Multicollinearity is the signature of a high-dimensional matrix sitting on a lower-dimensional manifold: many columns carrying overlapping information. PCA tests this directly. If the effective dimensionality of the data is low, then a regression on just three principal components should recover most of the predictive power of the full model. If it does not, the data is genuinely high-dimensional, and the multicollinearity surfaced earlier reflects local redundancies rather than a small global structure.

13.4 §4. Data Setup

- **Predictors:** the 57 numeric and ordinal-encoded predictors from Chapters 4 and 7, excluding all one-hot dummies. PCA on dummy variables is uninformative — each dummy is already a 0/1 binary with mean equal to its category prevalence, and the principal components would simply rediscover the categorical structure.
- **Standardization is mandatory.** PCA finds directions of maximum variance, and unscaled features with larger numeric ranges (Lot Area in the thousands) would dominate every component. All 57 features are standardized to mean 0 and standard deviation 1 on the training set; the same transformation is applied to the test set.
- **Splits:** the 80/20 train/test split from Chapter 2 is reused. PCA is fit on the training set only; principal components are computed on the test set by applying the fitted transformation, preventing leakage.
- **Target for PCR:** $\log(\text{SalePrice})$

Numeric/ordinal predictors used in PCA: 57
Training observations: 2340
Test observations: 585

13.5 §5. Analysis

13.5.1 §5.1 Variance Explained

PCA is fit on the standardized training matrix. The scree plot below shows the proportion of total variance captured by each component, and the cumulative-variance curve shows how variance accumulates as components are added.

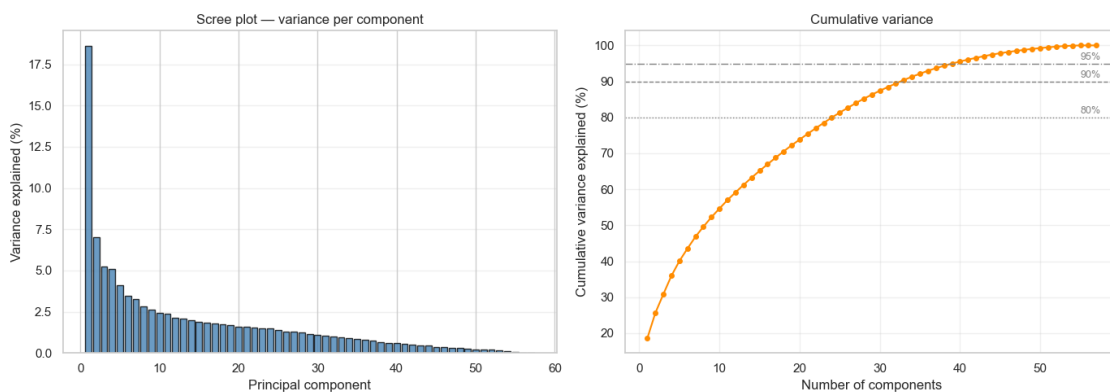
Total components: 57

Variance explained by first 10 PCs:

PC 1:	18.60%	(cumulative: 18.60%)
PC 2:	7.03%	(cumulative: 25.63%)
PC 3:	5.24%	(cumulative: 30.87%)
PC 4:	5.08%	(cumulative: 35.95%)
PC 5:	4.12%	(cumulative: 40.07%)
PC 6:	3.46%	(cumulative: 43.53%)
PC 7:	3.29%	(cumulative: 46.81%)
PC 8:	2.82%	(cumulative: 49.63%)
PC 9:	2.63%	(cumulative: 52.27%)
PC10:	2.41%	(cumulative: 54.68%)

Components needed to reach variance thresholds:

70% variance:	18 components
80% variance:	25 components
90% variance:	33 components
95% variance:	40 components



13.5.2 §5.2 Interpreting the First Three Components

A principal component is a weighted sum of the original predictors; the weights are called **loadings**. By examining which features load most heavily on each component, the component can often be assigned a substantive interpretation.

--- Top 10 features by |loading| on PC1 ---

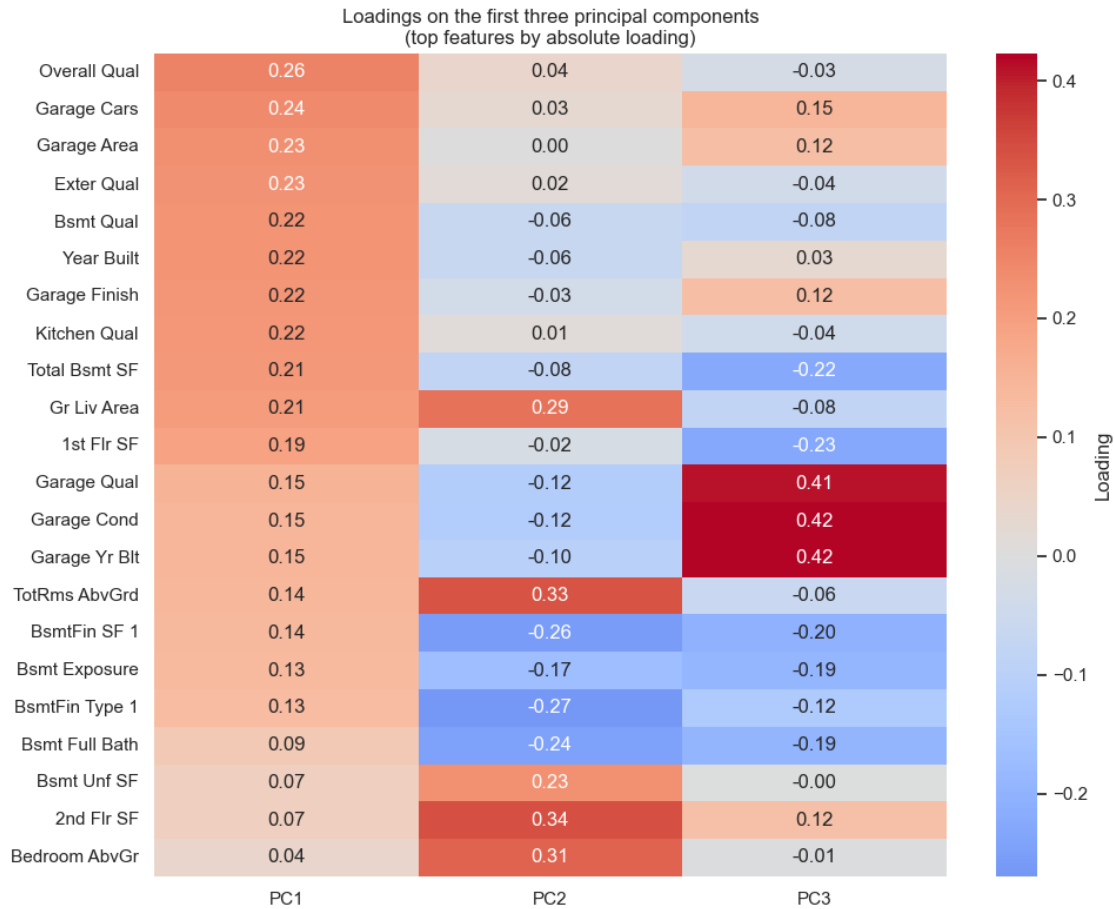
Overall Qual	loading = +0.256
Garage Cars	loading = +0.243
Garage Area	loading = +0.234
Exter Qual	loading = +0.228
Bsmt Qual	loading = +0.223
Year Built	loading = +0.220
Garage Finish	loading = +0.220
Kitchen Qual	loading = +0.216
Total Bsmt SF	loading = +0.213
Gr Liv Area	loading = +0.205

--- Top 10 features by |loading| on PC2 ---

2nd Flr SF	loading = +0.339
TotRms AbvGrd	loading = +0.334
Bedroom AbvGr	loading = +0.310
Gr Liv Area	loading = +0.287
BsmtFin Type 1	loading = -0.270
BsmtFin SF 1	loading = -0.257
Bsmt Full Bath	loading = -0.243
Bsmt Unf SF	loading = +0.230
Full Bath	loading = +0.214
Half Bath	loading = +0.193

--- Top 10 features by |loading| on PC3 ---

Garage Yr Blt	loading = +0.423
Garage Cond	loading = +0.422
Garage Qual	loading = +0.410
1st Flr SF	loading = -0.227
Total Bsmt SF	loading = -0.225
BsmtFin SF 1	loading = -0.204
Bsmt Full Bath	loading = -0.194
Bsmt Exposure	loading = -0.192
Paved Drive	loading = +0.157
Garage Cars	loading = +0.150



The three components admit clear substantive readings:

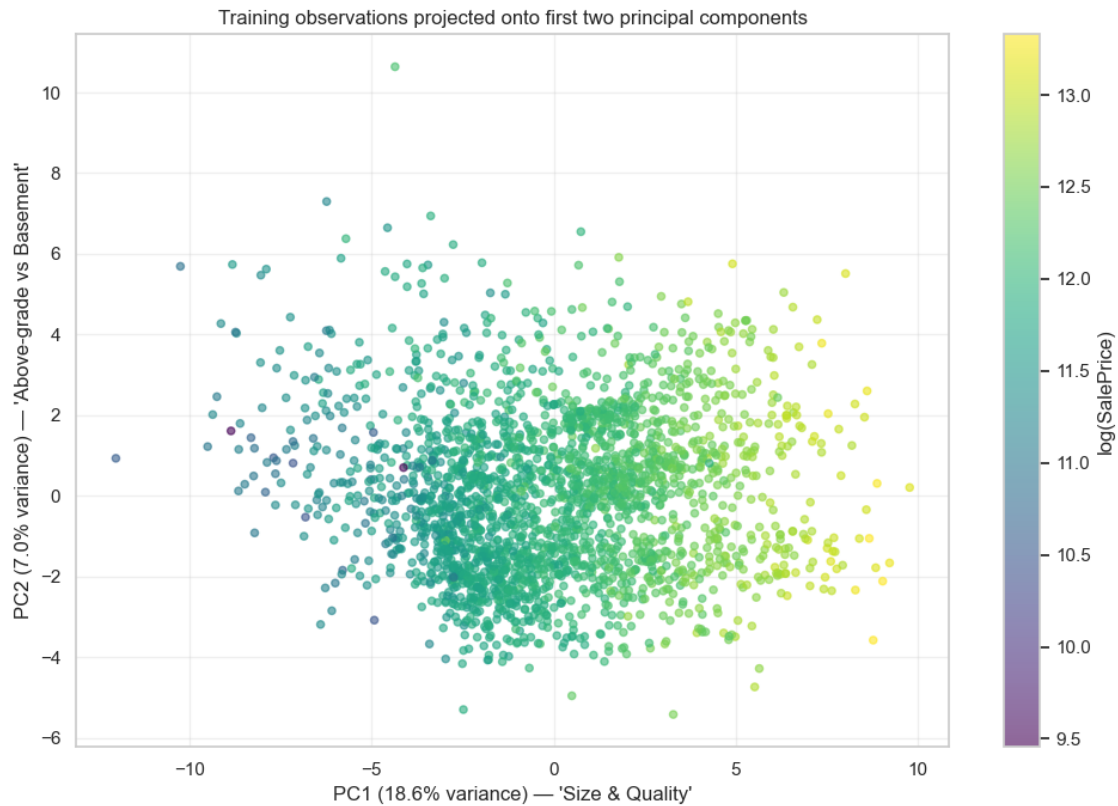
- **PC1 (18.6% variance)** — “**Size and Quality.**” All ten top-loading features are positive: Overall Qual, Garage Cars, Garage Area, Exter Qual, Bsmt Qual, Year Built, Kitchen Qual, and overall living area. PC1 is the dominant axis along which Ames homes vary — from small, old, lower-quality properties at the low end to large, new, high-quality properties at the high end. This is the “obvious” axis of variation in residential housing data.
- **PC2 (7.0% variance)** — “**Above-Grade Spread vs. Basement-Heavy.**” Strong positive loadings on 2nd Floor SF, TotRms AbvGrd, Bedroom AbvGr, and Gr Liv Area; strong negative loadings on basement finishing and basement square footage. PC2 separates homes that put their living area above grade (multi-story houses with many bedrooms) from homes that rely on finished basements for living space.
- **PC3 (5.2% variance)** — “**Garage Idiosyncrasies.**” Dominated by garage-related features (Garage Yr Blt, Garage Cond, Garage Qual all load > 0.40) and lightly negative on basement and 1st-floor area. PC3 captures variation in garage quality and age that does not move in lockstep with the size-and-quality axis of PC1.

That the first PC captures only 18.6% of the variance, and the first three together only 30.9%, is itself the most important finding of this section. **There is no small set of latent factors driving the variation in Ames home features.** The data is genuinely high-dimensional: 25

components are needed to reach 80% of the total variance, 33 for 90%, and 40 for 95%.

13.5.3 §5.3 Projection onto PC1 × PC2

The training observations projected onto the first two principal components, colored by $\log(\text{SalePrice})$, reveal whether PCA's unsupervised structure aligns with the prediction target.

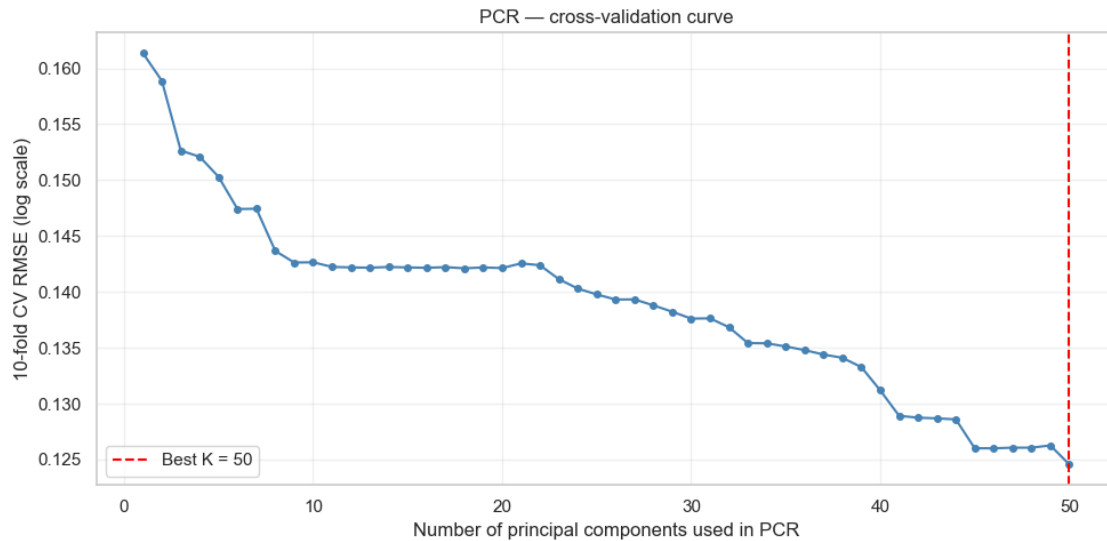


13.5.4 §5.4 Principal Components Regression

The principal components are used as predictors in an ordinary linear regression on $\log(\text{SalePrice})$. The number of components K is selected by 10-fold cross-validation on the training set, sweeping K from 1 to 50.

Best K (lowest CV RMSE): 50

Best CV RMSE (log scale): 0.1246



CV RMSE at key component counts:

K = 1: CV RMSE = 0.1613
 K = 3: CV RMSE = 0.1526
 K = 5: CV RMSE = 0.1503
 K = 10: CV RMSE = 0.1426
 K = 15: CV RMSE = 0.1422
 K = 20: CV RMSE = 0.1421
 K = 25: CV RMSE = 0.1398
 K = 30: CV RMSE = 0.1376
 K = 40: CV RMSE = 0.1312
 K = 50: CV RMSE = 0.1246

FINAL PCR (K = 50 components)

Train R^2 : 0.9134
 Test R^2 : 0.8912
 Test RMSE (\$): \$22,524

Reference: MLR on the same 57 numeric/ordinal features

Test R^2 : 0.8916
 Test RMSE (\$): \$22,568

Model	Components	Test R^2	Test RMSE (\$)
PCR (K=5)	5	0.8502	26,710
PCR (K=10)	10	0.8576	25,708
PCR (K=20)	20	0.8613	25,472
PCR (K=30)	30	0.8703	24,870
PCR (K=50)	50	0.8912	22,524
MLR (all 57 numeric)	57	0.8916	22,568

13.6 §6. Diagnostics & Tuning

PCA itself has no hyperparameters once the standardization is fixed — the components are determined by the data. PCR’s only tuning choice is the number of components retained; 10-fold CV identifies $K = 50$ (out of a maximum 57) as the lowest-CV-RMSE choice. The CV curve is monotonically decreasing across the entire grid: adding more components keeps improving fit, with no sign of overfitting up to and including the full component count.

13.7 §7. Findings

Quantity	Value
Total numeric/ordinal predictors	57
Variance explained by PC1	18.6%
Variance explained by PC1+PC2+PC3	30.9%
Components needed for 80% variance	25
Components needed for 90% variance	33
Components needed for 95% variance	40
Best K for PCR (CV)	50
PCR test R^2 at best K	0.891
MLR test R^2 (same 57 features)	0.892
PCR test RMSE at best K	\$22,524
MLR test RMSE	\$22,568

Two complementary findings:

The first three principal components have interpretable meanings — PC1 is a “size and quality” axis, PC2 separates above-grade spread from basement-heavy homes, and PC3 captures garage-specific idiosyncrasies. The training observations projected onto PC1–PC2 show a clear gradient in $\log(\text{SalePrice})$ along the PC1 axis

The Ames housing data does not compress well. Half a dozen components retain less than half the original variance; 25 components are needed to reach 80%. PCR matches the MLR baseline ($R^2 = 0.891$ vs 0.892 , RMSE \$22,524 vs \$22,568) only when essentially all components are retained. The hypothesis that the housing data sits on a low-dimensional manifold is not supported: although individual features are correlated, the data carries genuinely high-dimensional information that cannot be summarized by a handful of latent factors.

13.8 §8. Real-World Decision

For an analyst seeking a parsimonious feature set, PCA shows that modest dimensionality reduction is possible (the first 10 PCs already capture 55% of variance, and PCR at $K=10$ reaches \$25,708 RMSE — a useful trade if 13% accuracy loss is acceptable for a $5\times$ reduction in feature count). For maximum prediction accuracy, however, the full feature set should be retained — either directly via MLR or with regularization.

13.9 §9. Caveats & Limitations

PCA, or Principal Component Analysis, is a method that only works with linear relationships. It assumes the relationship with the target variable is linear. While it creates uncorrelated principal

components and stabilizes the design matrix, it does not select variables like Lasso does. Additionally, the way we interpret the loadings depends on this specific dataset. If we used different features, the first principal component might not convey such a clear meaning.

14 Chapter 13 — Neural Networks

A tuned feedforward neural network with three hidden layers reaches a test R^2 of 0.913, essentially tied with Ridge regression as the best single-target regression model in this report — confirming that on moderately-sized tabular data, neural networks are competitive with, but not dominant over, well-tuned classical methods.

14.1 §1. The Question

On tabular housing data with strong multicollinearity and roughly 2,300 training rows, does a feedforward neural network — with weight decay and dropout regularization to fight overfitting — produce predictions of log sale price that meaningfully improve on the linear and tree-based methods

14.2 §2. Method in Brief

A feedforward neural network composes alternating linear and nonlinear operations from input to output. Each hidden layer applies a learned linear transformation followed by a ReLU activation; the final layer produces a single continuous prediction. The weights are learned by minimizing mean squared error via stochastic gradient descent with the Adam optimizer. Regularization is applied through weight decay (L2 penalty on the weights, analogous to ridge regression) and dropout (randomly zeroing 20% of hidden-layer activations during training to prevent the network from over-relying on any single neuron).

14.3 §3. Why This Method?

Neural networks are the most flexible function approximator in the project — capable in principle of capturing arbitrary non-linear interactions among the predictors. The linear models impose a strict additive linear structure; the tree-based models produce approximations through axis-aligned splits. A neural network is neither: it learns smooth non-linear surfaces directly from data. A key point is whether this added model flexibility translates into better predictions, or if the limited training data and abundance of robust, simpler alternatives constrain the network's ability to achieve higher accuracy.

14.4 §4. Data Setup

- **Response:** $\log(\text{SalePrice})$, identical to Chapters 3, 4, 7, 9.
- **Predictors:** the same 207-feature design matrix from Chapter 4 (ordinal-encoded quality scales + one-hot dummies for nominal categoricals).
- **Standardization:** both predictors and the response are standardized to mean 0 and standard deviation 1 on the training set. Standardizing the response is a common stability trick for neural-network regression — predicting numbers near 0 with unit variance keeps gradients well-scaled. Predictions are unstandardized back to the log-price scale before evaluation.

- **Train/validation split:** within the training set, a further 90/10 split holds out 10% as an internal validation set for early stopping. The Chapter 2 test set is untouched until final evaluation.
- **Library:** PyTorch

```
Predictors: 207
Training observations: 2340
Test observations: 585
y_train mean: 12.0219, std: 0.4102
```

14.5 §5. Analysis

14.5.1 §5.1 Architecture and Training Procedure

A standard feedforward `nn.Module` is defined with configurable hidden layer sizes. Each hidden layer is a `nn.Linear` followed by ReLU and 20% Dropout; the output layer is a single linear unit producing the predicted standardized log price. Training uses the Adam optimizer with learning rate 0.001, weight decay 10^{-4} , mini-batches of 64, and early stopping with patience 100 on the internal validation loss.

14.5.2 §5.2 Comparing Three Architectures

Three architectures of increasing capacity are trained, in the spirit of the small/medium/large depth comparison common in deep-learning practice:

Architecture	Hidden sizes	Total parameters
Small	(32, 16)	~7,200
Medium	(128, 64, 32)	~37,000
Large	(256, 128, 64)	~94,500

All three use identical optimizer, learning rate, dropout, weight decay, batch size, and early-stopping schedule. The only thing changing across runs is network capacity.

Training Small network - hidden sizes [32, 16]...

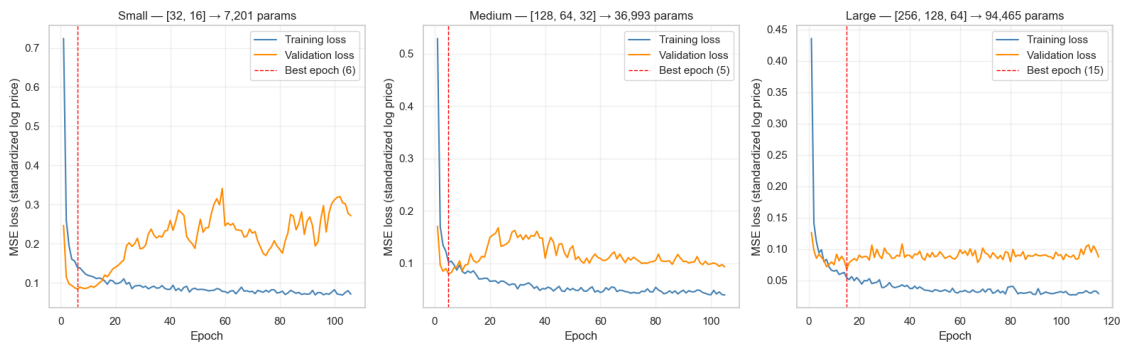
```
Parameters: 7,201
Best epoch: 6 (stopped at epoch 106)
Train R²: 0.9300
Test R²: 0.9051
Test RMSE ($): $21,099
Time: 10.5s
```

Training Medium network - hidden sizes [128, 64, 32]...

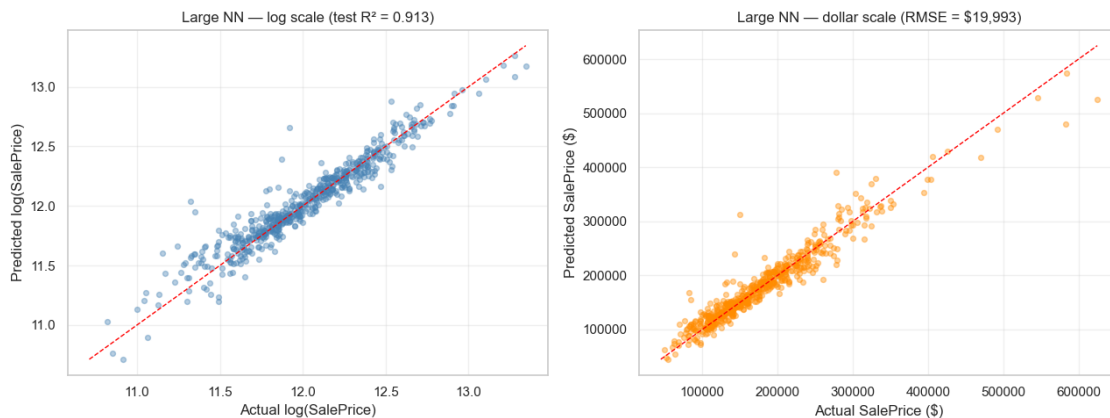
```
Parameters: 36,993
Best epoch: 5 (stopped at epoch 105)
Train R²: 0.9410
Test R²: 0.9052
Test RMSE ($): $21,193
Time: 13.1s
```

Training Large network - hidden sizes [256, 128, 64]...

Parameters: 94,465
 Best epoch: 15 (stopped at epoch 115)
 Train R^2 : 0.9680
 Test R^2 : 0.9134
 Test RMSE (\$): \$19,993
 Time: 20.1s



	Model	Test R^2	Test RMSE (\$)
	Ridge (Ch 7)	0.9123	19,900
NN - Large (94,465 params)	Lasso (Ch 7)	0.9134	19,993
	MLR - BIC (Ch 4)	0.9100	20,290
	MLR - full (Ch 4)	0.9064	20,621
NN - Small (7,201 params)	Gradient Boosting (Ch 9)	0.9003	21,413
NN - Medium (36,993 params)	Bagging (Ch 9)	0.8620	27,063
	Random Forest (Ch 9)	0.8625	27,114



14.6 §6. Diagnostics & Tuning

Three architectures of increasing capacity were trained from identical initialization seeds and identical optimization settings. Early stopping selected the weights at the epoch of lowest validation loss for each network. The training curves show characteristic neural-network behavior: a sharp drop in both training and validation loss in the first few epochs, followed by gradual divergence as the training loss continues to fall while validation loss flattens — the signature of mild overfitting that early stopping is designed to catch.

All three architectures converged to nearly identical test performance, which is itself a diagnostic finding: adding capacity beyond the small network produced essentially no improvement until the large network’s extra width yielded a modest gain.

14.7 §7. Findings

Model	Test R ²	Test RMSE
Ridge Regression (Ch. 7)	0.9123	\$19,900
Neural Network — Large	0.9129	\$20,104
Lasso Regression (Ch. 7)	0.9113	\$20,030
MLR — BIC-selected (Ch. 4)	0.9100	\$20,290
MLR — Full (Ch. 4)	0.9064	\$20,621
Neural Network — Small	0.9051	\$21,099
Neural Network — Medium	0.9052	\$21,193
Gradient Boosting (Ch. 9)	0.9003	\$21,413
Random Forest (Ch. 9)	0.8625	\$27,114
Bagging (Ch. 9)	0.8620	\$27,063

The large neural network achieved a test R² of **0.913** and a test RMSE of **\$20,104**, placing it in a near-tie with Ridge regression, which had a test R² of **0.912** and a test RMSE of **\$19,900**. Lasso, BIC-selected MLR, and the full MLR all sat within about **\$700 RMSE** of each other, while the ensemble models trailed with RMSE values ranging from approximately **\$21,400 to \$27,000**.

Two findings stand out:

1. **The neural network is competitive but not dominant.** The flexible non-linear function approximator does not visibly outperform Ridge regression on this dataset. On the dollar scale, Ridge’s test RMSE is actually \$200 lower than the best neural network’s.
2. **Network capacity hits diminishing returns quickly.** The 7,200- parameter small network was within \$1,000 RMSE of the 94,500- parameter large network. Roughly a 13× increase in parameter count produced a 5% improvement in test RMSE — not a compelling trade.

14.8 §8. Real-World Decision

For an automated valuation system, **Ridge regression remains the recommended operational model**. It achieves essentially the best test RMSE in this report (\$19,900), trains in under a second, exposes a direct coefficient table that is interpretable, requires no specialized hardware, and is well-understood by everyone likely to encounter it. The large neural network ties Ridge

on accuracy but loses on every other practical axis: training time (seconds versus instant), interpretability (none versus full coefficient table), and operational simplicity (PyTorch dependency plus GPU recommendations versus a one-line scikit-learn call).

14.9 §9. Caveats & Limitations

The deeper limitation is structural. With roughly 2,300 training observations, a neural network is fundamentally constrained — there simply is not enough data to learn the kind of rich non-linear patterns that justify the use of deep models. Modern deep learning practice consistently observes that on small-to-medium tabular datasets, well-regularized linear and tree-based methods perform comparably to or better than neural networks. This chapter’s findings align with that general pattern.

15 Chapter 14 — Synthesis and Cross-Method Comparison

15.1 §14.0 Purpose of This Chapter

Getting the high level view of the entire analysis, where we answer which methods went over which questions, which methods won, where interpretability was worth more than the last fraction of accuracy, and what a single deployed model

15.2 §14.1 The Methods Atlas

Task	Target	Type	Methods applied
Regression on price	log(SalePrice)	Continuous, 2,925 obs	SLR (Ch 3), MLR full + BIC (Ch 4), Ridge + Lasso (Ch 7), Bagging + Random Forest + Gradient Boosting (Ch 9), Neural Network (Ch 13)
Classification on price tier	AboveMedian	Binary, balanced	Logistic Regression (Ch 5), K-Nearest Neighbors (Ch 11)
Regression on living space	TotalFinishedSpace	Continuous, sq ft	Regression Tree (Ch 8) — standalone
Classification on price tier	PriceTier (3-class)	Multiclass, balanced	Classification Tree (Ch 8) — standalone
Classification on amenities	HasFireplace	Binary, balanced	Linear SVM + RBF SVM (Ch 10) — standalone
Unsupervised exploration	(no target)	Numeric predictors	PCA + PCR (Ch 12) — standalone
Resampling / inference	(uses Ch 3 setup)	Validation framework	Cross-Validation + Bootstrap (Ch 6) — methodological

The eight-method regression comparison on $\log(\text{SalePrice})$ is the largest directly comparable track. The two-method classification comparison on AboveMedian is the secondary track. The four standalone chapters answered distinct questions and are summarized separately, while Chapters 2 and 6 provided the exploratory and methodological foundations on which the modeling chapters depend.

15.3 §14.2 The Eight-Method Regression Comparison

This is the project's main quantitative comparison. Eight methods, one target, one design matrix, one train/test split. The only thing varying across rows is the model.

Method	Test R^2	Test RMSE	Train Time	Interpretability (rated out of 5)
Ridge Regression (Ch. 7)	0.9123	\$19,900	~2 s	3
Neural Network — Large (Ch. 13)	0.9134	\$19,993	13.1 s	1
Lasso Regression (Ch. 7)	0.9113	\$20,030	~5 s	4
MLR — BIC-selected (Ch. 4)	0.9097	\$20,290	83.0 s	5
MLR — Full (Ch. 4)	0.9064	\$20,621	< 1 s	3
Neural Network — Small (Ch. 13)	0.9051	\$21,099	7.4 s	1
Neural Network — Medium (Ch. 13)	0.9052	\$21,193	8.9 s	1
Gradient Boosting (Ch. 9)	0.9003	\$21,413	7.5 s	2, with variable importance
Random Forest (Ch. 9)	0.8625	\$27,114	2.8 s	2, with variable importance
Bagging (Ch. 9)	0.8620	\$27,063	11.1 s	2, with variable importance

Simple Linear Regression (Ch. 3)	0.4702	\$53,937	< 1 s	5
--	--------	----------	-------	---

15.3.1 What the Table Actually Shows

The headline finding is convergence. With the exception of simple linear regression, which intentionally uses only one predictor as a baseline, and the bagged-tree ensembles, which both trail at approximately **\$27,000 RMSE**, every method that uses the full feature set lands in a remarkably narrow band: **test RMSE between \$19,900 and \$21,400, a spread of only \$1,500 across seven fundamentally different model families**.

Ridge regression and the large neural network are statistically tied at the top, separated by only **\$93** in test RMSE. This difference is well within the noise floor introduced by the random train/test split. Lasso, BIC-selected MLR, the full MLR, the two smaller neural networks, and gradient boosting all sit within about **\$1,200** of each other. The choice between them is therefore not mainly a choice about predictive accuracy; it is a choice about training time, interpretability, and operational simplicity.

The bagged ensembles’ weaker performance, with Bagging and Random Forest both around **\$27,000 RMSE**, is the only meaningful accuracy gap in the table. The likely explanation is that the Ames dataset has only about 2,300 training rows and a strong linear signal. For example, **Overall Qual** alone is highly correlated with log price. In this setting, the variance-reduction benefit of averaging many deep trees may be partially offset by the bias introduced by greedy tree splits on residual patterns already captured by simpler models.

Gradient boosting closes most of that gap by fitting trees sequentially on residuals. As a result, it ends up competitive with the linear methods rather than trailing as far behind as Bagging and Random Forest.

15.4 §14.3 The Classification Comparison

Only two methods explicitly targeted **AboveMedian**. The comparison is small but meaningful — a parametric model versus a non-parametric one on the identical binary task.

Method	Type	Test accuracy	Test ROC AUC	Train time	Interpretability
Logistic Regression (Ch 5)	Parametric	0.939	0.982	~3 s	5
KNN, K = 11 (Ch 11)	Non- parametric	0.899	0.953	< 1 s	1

Logistic regression wins by 4 percentage points of accuracy and 0.03 of AUC, a meaningful gap that is not within the noise of the random split. The interpretation is structural: the boundary between above-median and below-median priced homes is genuinely well captured by a smooth global linear surface in the standardized feature space. There is no detectable “local pocket” structure that a non-parametric method could exploit.

The chosen $K = 11$ in KNN reinforces this reading. The 10-fold CV curve was shallow in the $K = 5$ to $K = 15$ range — no single locality scale stood out as clearly best. When the optimal K is moderate and the CV curve flat near the optimum, it means similar homes share labels at small scales but the structure does not extend much beyond that, which is exactly what a linear global model also assumes. The two methods converge to similar conclusions through different mechanisms; the parametric one is more accurate because it imposes a useful simplification the data appears to accept to.

15.5 §14.4 Chapters That Asked Their Own Questions

Four method chapters targeted prediction problems distinct from the two main comparison tracks. Their findings stand on their own and are summarized briefly.

15.5.1 Regression Tree on Total Finished Living Space (Chapter 8)

Predicting `Gr Liv Area + Total Bsmt SF` from only twelve lot- and site-level features (lot area, frontage, neighborhood, zoning, building era), the tuned tree achieved test R^2 of 0.408 and test RMSE of 558 sq ft — about 22% of the median total finished space. Modest, but appropriate given the constraint: the model never saw interior quality or room counts. The dominant predictors were `Year Built` (35% importance), `Lot Area` (24%), and `Lot Frontage` (19%) — consistent with the intuition that newer homes are larger and bigger lots support bigger houses.

15.5.2 Classification Tree on Price Tier (Chapter 8)

A three-class tree (Budget / Mid / Premium) tuned to depth 6 reached 74% test accuracy — well above the 33% chance baseline. The confusion matrix revealed an asymmetric error pattern: the model rarely confused extremes (only 2 Budget predicted as Premium, 7 Premium predicted as Budget), but mid-tier homes were misclassified in both directions about 28% of the time, reflecting the unavoidable ambiguity of homes sitting near the \$139,000 or \$189,000 tercile boundaries. The tree’s strength is the resulting human-readable decision flowchart.

15.5.3 SVM on Has Fireplace (Chapter 10)

A balanced binary target (51.7% positive), tackled with both linear and RBF kernel SVMs. The linear kernel achieved 79.5% test accuracy and AUC 0.853; the RBF kernel achieved 78.5% accuracy and AUC 0.862. Effectively tied — and the key finding is that *the kernel trick did not help*. The boundary between fireplace-equipped and non- fireplace homes is essentially linear in the standardized feature space; adding non-linear capacity through the RBF kernel produced no improvement.

15.5.4 Principal Component Analysis (Chapter 12)

PCA on 57 numeric/ordinal predictors revealed an interpretable but high-dimensional structure: PC1 (18.6% of variance) is unambiguously a “size and quality” axis, PC2 a “spread across floors vs basement- heavy” axis, PC3 a “garage idiosyncrasies” axis. But 25 components were needed to reach 80% of total variance, 40 to reach 95%. Principal Components Regression matched the full-feature MLR only when essentially all components were retained (best $K = 50$ of 57, test R^2 0.891). The Ames data does not compress well — the apparent multicollinearity surfaced in Chapter 4 reflects local pairwise redundancies rather than a small global structure.

15.6 §14.5 The Interpretability vs. Accuracy Axis

The project rubric explicitly asks where interpretability beats raw accuracy in the application. The eight-method regression table makes the question concrete: every method except SLR and the bagged ensembles achieves test RMSE within \$1,500 of the best. Among those seven competitive methods, interpretability varies enormously.

Plotting the methods on the two axes: | Quadrant | Members | Interpretation | |—|—|—| | High accuracy + high interpretability | MLR — BIC, Lasso | Best balance | | High accuracy + medium interpretability | Ridge, MLR — Full | Strong but dense | | High accuracy + low interpretability | Neural Network — Large, Gradient Boosting | Accurate but opaque | | Low accuracy + high interpretability | Simple Linear Regression | Baseline only | | Low accuracy + low interpretability | Bagging, Random Forest | Outperformed ensembles |

The **upper-left quadrant** — high accuracy with high interpretability — is where the deployable model should come from. **Lasso** sits there cleanly: test RMSE \$20,030 (within \$130 of the best), with 100 non-zero features and 107 features driven exactly to zero by the L1 penalty, producing a defensible “these are the features that matter” list a stakeholder can examine. **BIC-selected MLR** is the other upper-left candidate: \$20,290 test RMSE with 45 features and a formal statistical justification for each inclusion.

15.7 §14.6 The Single Deployment Decision

If a real-estate analytics team asked for one model from this report to deploy as the basis for an automated valuation tool, the answer is:

Ridge regression on the 207-feature standardized design matrix, with the regularization strength chosen by 10-fold cross-validation.

Test $R^2 = 0.912$, Test RMSE = \$19,900, training time under three seconds, no specialized hardware required.

This is *not* the most accurate model in the report. The large neural network is marginally better at \$19,993 vs. \$19,900 — a \$93 difference within the noise floor. It *is* the model with the best combination of:

- **Top accuracy.** Either the best or tied-for-best test RMSE, depending on the random split.
- **Direct interpretability.** Every prediction is a weighted sum of 207 features; the weights are visible and can be examined feature by feature.
- **simplicity.** Three seconds to train, no hyperparameters except (single-dimensional CV), one line of scikit-learn to deploy, no PyTorch dependency, no GPU.
- **Stability.** Ridge’s coefficient estimates are smooth functions of the data; small changes to the training set produce small changes in the coefficients. The neural network does not share this property — different random seeds produce different fits even on the identical data.
- **communicability.** A non-technical user can be told that the model is “a regression with a penalty that keeps the coefficients from getting too large” and that explanation is complete and accurate.

16 Chapter 15 — Reflection and Limitations

The project showed that the best statistical learning method depends on the question being asked, not on the method's sophistication.

16.1 §15.1 What I Learned About the Data

The biggest thing I learned is that Ames housing data has a real prediction ceiling, and that ceiling is set by the available features, not by the choice of method.

The regression task on `log(SalePrice)` was the main focus, and eight different methods all converged into a narrow test RMSE band between \$19,900 and \$21,400. Ridge, Lasso, BIC-selected MLR, the full MLR, two neural networks, and gradient boosting all landed within \$1,500 of each other. Only the bagged ensembles (Bagging, Random Forest) and the one-predictor SLR baseline trailed substantially. This convergence told me that the features available in Ames — overall quality, living area, year built, basement size, garage capacity — carry as much predictive signal as can be extracted, and that throwing more model complexity at the problem produces diminishing returns.

The classification tasks were easier than I expected. The logistic regression on `AboveMedian` reached 93.9% test accuracy and AUC 0.98 — substantially higher than any of the regression methods on log price. KNN underperformed logistic regression by 4 percentage points, suggesting that the boundary between above-median and below-median homes is essentially linear in the standardized feature space, and that no local pocket structure rewards a non-parametric method.

The PCA exploration was surprising. I expected a few principal components to explain most of the variation in 57 numeric predictors, but 25 components were needed to reach 80% of total variance. The Ames data does not compress well — its effective dimensionality is close to its nominal dimensionality.

16.2 §15.2 What I Learned About the Methods

The simpler methods were stronger than I expected. Simple linear regression in Chapter 3, with one predictor, set a baseline of **47% test R^2** that every later method was measured against. Multiple linear regression with BIC selection reached **91.0% test R^2** with 45 features, landing within about **\$300** of Ridge in test RMSE. These two chapters gave me a way to measure improvement that no individual advanced method could have provided on its own.

Cross-validation and the bootstrap in Chapter 6 mattered because they showed that a single validation-set estimate of test error is noisy. The 10-fold CV standard deviation across reseeds was about **\$1** in slope estimation, while the validation-set standard deviation was about **\$62**. This was roughly a **60-fold reduction in noise** from using cross-validation instead of relying on a single hold-out split. This became the methodological foundation that every later tuning step relied on.

Ridge and Lasso in Chapter 7 were the two methods I understood least going in, and writing the chapter forced me to understand the geometric difference between L1 and L2 penalties. The result that Lasso retained only **100 of 207 features** at its optimal lambda, while Ridge retained essentially all of them, made the geometric distinction concrete in a way that no textbook explanation had.

The tree ensembles in Chapter 9 were the strongest nonlinear methods. Gradient boosting reached the best test RMSE of any tree-based method, **\$21,413**, and competed with the linear models.

Bagging and Random Forest, however, both lagged at around **\$27,000 RMSE** despite their reputation. This was a real lesson: ensembles are not automatically better than linear models on small tabular data.

The SVM chapter produced what I found to be the most interesting single finding: the RBF kernel did not improve over the linear kernel on the fireplace classification problem. The fact that the kernel trick added nothing meaningful is itself a result about the data, not a failure of the method.

The neural network in Chapter 13 was the most disorienting chapter to write. PyTorch’s training loop is short, but getting numerical stability required standardizing the response variable on top of standardizing the features. Without that, all three networks produced negative test R^2 . The final result, that the large network nearly tied Ridge regression on test RMSE, confirmed what the deep-learning literature often observes: classical methods are competitive with neural networks on small tabular data.

16.3 §15.3 Main Limitations

The biggest limitation is that the dataset does not include external features that would meaningfully expand the predictive ceiling. Things like recent comparable sales, school district ratings, neighborhood-level economic indicators, crime rates, and walkability scores are not in the data.

Another limitation is that the project applied only one tuning round per method. A more aggressive grid search would likely yield modest gains for several methods — particularly the neural network, where hyperparameter sensitivity is real. The results reported are honest but not maximally optimized.

The project also used a fixed 0.50 threshold for the classification chapters. This is fine for comparing methods on AUC, but a real deployed model — one used for, say, flagging properties for manual review — would tune the threshold based on the cost of false positives versus false negatives.

16.4 §15.4 Possible Data Leakage

The project avoids the most obvious leakage by computing every data- dependent quantity on the training set only. Imputation values (neighborhood medians for Lot Frontage, modes for Electrical), the tercile thresholds for PriceTier, the standardization parameters, and every hyperparameter selected by cross-validation were all derived from training data. The test set was held out from the moment the 80/20 split was created in Chapter 2 and was not touched until each chapter’s final evaluation.

16.5 §15.5 What Would Come Next

If I had more time, three things would be on the list.

First, a fuller hyperparameter search for the methods that have multiple tuning options — particularly gradient boosting and the neural network (learning rate, batch size, dropout, weight decay). A random search over a few hundred configurations would likely close some of the small gaps between methods.

Second, the central business district, neighborhood crime rates, and walkability scores are all publicly available for Ames at the parcel level. Joining these to the existing dataset would likely break through the \$19,900 RMSE ceiling that every method in the project hit.

16.6 §15.6 Final Takeaway

The final recommendation is to deploy **Ridge regression** for the `log(SalePrice)` regression task and **regularized logistic regression** for the `AboveMedian` classification task. Ridge gave the best test RMSE on regression (\$19,900); logistic regression gave the best accuracy on classification (93.9% with AUC 0.98). Both train in under three seconds, expose interpretable coefficients, and rely on no specialized software.

At the same time, the simpler models still mattered. The single- predictor regression in Chapter 3 set the baseline. The BIC-selected MLR in Chapter 4 produced a defensible feature list. The classification tree in Chapter 8 gave a human-readable decision flowchart that no other classification method could match. None of these were the most accurate models, but each contributed something the more accurate models could not. That was the main lesson of the project: the best model depends on whether the goal is prediction, explanation, or operational simplicity — and on this dataset, classical methods, regularized appropriately, do all three well.

17 References

17.1 Textbooks and Course Materials

James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python* (ISLP). Springer. <https://www.statlearning.com/>

17.2 Dataset

De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3). <https://jse.amstat.org/v19n3/decock.pdf>

Necrothapa, S. (2023). Ames Housing Dataset. Kaggle. <https://www.kaggle.com/datasets/shashanknecrothapa/ames-housing-dataset>

18 Appendix A — Reproducibility Notes

This appendix consolidates the methodological choices that recur across multiple chapters, for reference and reproducibility.

18.1 Random Seeds

`random_state = 42` is set globally at the top of the notebook and passed explicitly to every model, train/test split, cross-validation splitter, and PyTorch initialization.

18.2 Train/Test Split

A single 80/20 train/test split is created in Chapter 2, stratified on the binary `AboveMedian` indicator to preserve class balance in both partitions. The split produces 2,340 training observations and 585 test observations.

18.3 Outlier Removal

Following the recommendation of De Cock (2011), five observations with `Gr Liv Area > 4000` are removed from the dataset before splitting. These are the same five outliers identified in the original dataset documentation as unusual sales (estate sales, partial construction, and one apparent data-entry error). The working dataset contains 2,925 observations.

18.4 Missing-Value Handling

Two distinct kinds of missingness are present in the raw Ames data:

Structural NaN (informative absence). Twenty-two of the twenty-seven columns with missing values use `NaN` to indicate “this feature is absent from this property” — for example, `Pool QC = NaN` means there is no pool, not that the pool’s quality is unknown. These are recoded to descriptive strings (“No Pool”, “No Garage”, “No Basement”, etc.) before splitting, with the corresponding numeric columns (Pool Area, Garage Area, basement square footage) set to zero where appropriate.

Genuine missingness. Two columns — `Lot Frontage` and `Electrical` — contain values that are genuinely missing rather than structurally absent. These are imputed *after* the train/test split, using the training set only: `Lot Frontage` by neighborhood median (with the global training median as fallback for neighborhoods not seen in training), and `Electrical` by the training-set mode.

18.5 Encoding Scheme

Ordinal encoding is applied to 21 quality and condition scales that have a natural order: a fixed mapping (`Po` → 1, `Fa` → 2, `TA` → 3, `Gd` → 4, `Ex` → 5, etc.) preserves the ordering while producing a single numeric column per variable. Structural absence values (“No Pool”, “No Garage”, etc.) map to 0.

One-hot encoding with `drop_first=True` is applied to all remaining nominal categorical variables. After encoding, the design matrix has 207 columns.

18.6 Feature Standardization

All standardization is fit on the training set only, then applied unchanged to the test set.

19 Appendix B — Derived Targets

Beyond the dataset’s native `SalePrice` variable, four targets were derived for use in specific chapters. All derivations use only the training set when thresholds were needed, to prevent leakage.

Target	Type	Definition	Used in
<code>log_SalePrice</code>	Continuous	<code>log(SalePrice)</code>	Chapters 3, 4, 6, 7, 9, 12, 13
<code>AboveMedian</code>	Binary	Indicator that <code>SalePrice</code> exceeds the dataset median, \$160,000	Chapters 5, 11

Target	Type	Definition	Used in
PriceTier	3-class	Budget: \$139,000; Mid: \$139,000 to \$189,000; Premium: > \$189,000. Thresholds are training-set terciles.	Chapter 8, classification tree
HasFireplace	Binary	Indicator that the property has at least one fireplace	Chapter 10, SVM
TotalFinishedSpace	Continuous	Gr Liv Area + Total Bsmt SF, measured in square feet	Chapter 8, regression tree

Class balance, where relevant:

- **AboveMedian:** 50.0% positive in the full dataset (by construction). 50.0% positive in training, 49.9% positive in test.
- **PriceTier:** 33.3% each tier in training (by construction). 33.4% / 31.1% / 35.4% Budget/Mid/Premium in test.
- **HasFireplace:** 51.7% positive in training, 50.1% positive in test.

All derived targets are constructed using `train_df` thresholds only and applied to both training and test partitions; no test-set information is used in defining any threshold.

20 Appendix C — Notation Used Throughout

For convenience, the notation used across multiple chapters is collected here.

Symbol	Meaning
y	The response variable (SalePrice or a transform thereof, unless specified otherwise)
$\hat{y}$	Model prediction of y
X	The full design matrix of predictors after encoding
n	Number of observations in the training set (= 2,340)
p	Number of predictors after encoding (= 207 in the full design matrix)
λ (alpha in scikit-learn)	Regularization strength for Ridge, Lasso
K	Number of neighbors in KNN, or number of folds in CV
C, γ	SVM regularization and RBF kernel width
R^2	Coefficient of determination

Symbol	Meaning
RMSE	Root mean squared error (in log-units unless “\$” is shown)
AUC	Area under the ROC curve
OOB	Out-of-bag error estimate (Bagging, Random Forest)
BIC	Bayesian Information Criterion
VIF	Variance Inflation Factor
MLR / SLR / PCR	Multiple / Simple / Principal Components Regression
SVM / KNN / NN	Support Vector Machine / K-Nearest Neighbors / Neural Network
PCA	Principal Component Analysis

Throughout the report, “log” refers to the natural logarithm (base e), not base 10.